



République tunisienne

Ministère de l'Enseignement supérieur
et de la Recherche scientifique

École Privée Supérieure d'Ingénierie et de Technologie

TEK-UP



RAPPORT DE PROJET DE FIN D'ÉTUDES

en accomplissement partiel des exigences pour l'obtention du diplôme de

Diplôme National d'Ingénieur en Sciences Appliquées et Technologie

Specialization : Génie logiciel et systèmes d'information

Préparé par

Mohamed Aziz Merai

Plateforme intelligente de recommandation et d'optimisation de L'infrastructure IT

Encadrant professionnel : **Monsieur Mehdi Kaouech**

Encadrant académique : **Madame Inès AGREBI**

CEO RedSys

Enseignante encadrante

— TEK-UP University

J'autorise l'étudiant à soumettre son rapport de stage en vue de la soutenance.

Professional Supervisor, **Mr. Mehdi Kaouech**

Signature and Stamp

J'autorise l'étudiant à soumettre son rapport de stage en vue de la soutenance..

Academic Supervisor, **Mme. Ines Agerbi**

Signature

Dédicace

À mes parents,

les deux seules personnes qui me connaissent depuis mon premier souffle

Vous avez tout donné à un seul enfant—

vos rêves, vos sacrifices, vos nuits blanches, vos prières silencieuses.

Vous m’avez offert le monde, sans rien demander en retour, si ce n’est mon bonheur.

À mes amis,

les frères et sœurs que je n’ai jamais eus.

Vous avez comblé le silence, partagé le poids,
et fait en sorte que j’aie toujours quelqu’un à appeler.

Vous êtes ma famille de cœur, mon rire, mon refuge.

À mes parents, encore — parce que vous méritez d’être cités deux fois,

chaque page de ce travail porte votre nom à l’encre invisible.

Chaque réussite est la vôtre.

Chaque pas en avant est possible parce que vous ne m’avez jamais laissé marcher seul. *À ceux*

qui sont partis trop tôt,

vous n’êtes pas là pour tenir ce diplôme avec moi,
mais je vous porte dans chaque battement de mon cœur et dans chaque victoire.

Ce travail est autant le vôtre que le mien.

Ceci est pour vous.

Remerciements

Avec une profonde gratitude...

Je tiens à exprimer ma plus profonde et sincère gratitude à toutes les personnes qui, par leurs conseils, leur soutien et leurs encouragements, ont contribué à la réussite de ce projet.

Tout d'abord, j'adresse mes plus vifs remerciements à mon encadrant professionnel,

Monsieur Mehdi Kaouech

dont l'accompagnement précieux, les remarques constructives et le soutien constant ont été une véritable source d'inspiration tout au long de ce stage. Son expertise et son engagement ont grandement enrichi ce travail.

Je suis également profondément reconnaissant envers mon encadrant académique,

Madame Inès AGREBI

pour son suivi attentif, ses encouragements constants et ses précieux conseils académiques. Sa patience et sa confiance en mes capacités ont été très motivantes.

J'adresse également mes sincères remerciements à toute l'équipe de

RedSys

pour leur accueil chaleureux, leur excellente collaboration et leur aide continue durant mon stage. L'environnement convivial et stimulant qu'ils ont offert a rendu cette expérience à la fois enrichissante et mémorable.

Table des matières

Liste des abréviations	ix
Introduction générale	1
1 Contexte et Objectifs du Projet	3
1.1 Introduction	4
1.2 Présentation de l'organisme d'Accueil	4
1.2.1 Présentation de la Société RedSys	4
1.2.2 Mission et Vision de l'Entreprise	4
1.2.3 Organigramme	5
1.3 Présentation du projet	5
1.3.1 Etude de l'existant	5
1.3.2 Critique de l'existant	6
1.3.3 Solution proposée	6
1.4 Présentation de la méthodologie de gestion de projet	7
1.4.1 Les roles SCRUM	8
1.4.2 Les événements Scrum	8
1.4.3 Les artefacts de Scrum	9
1.5 Présentation du langage de modélisation	10
1.6 Conclusion	11
2 Analyse et spécification des besoins	12
2.1 Introduction	13
2.2 Spécification des besoins	13
2.2.1 Identification des acteurs	13
2.2.2 Spécification des besoins fonctionnels	13
2.2.3 Spécification des besoins non fonctionnels	15
2.3 Diagramme de cas d'utilisation global	15
2.4 Diagramme de Classes	16
2.5 Structure et découpage du projet	17
2.5.1 Identification de l'équipe SCRUM	17
2.5.2 Backlog de produit	17

2.5.3	Planification des Sprints	22
2.6	Architecture de l'Application	25
2.6.1	Architecture Physique	25
2.6.2	Architecture Logique	26
2.7	Choix Techniques et Environnement de Développement	27
2.7.1	Langages de Programmation	27
2.7.2	Couche API	29
2.7.3	Frameworks et Bibliothèques	29
2.7.4	Base de Données et ORM	31
2.7.5	DevOps et Déploiement	32
2.7.6	Outils de Développement	34
2.8	Conclusion	36
3	Release 1 : MVP	37
3.1	Introduction	38
3.2	Backlog de la Release 1	38
3.3	Diagramme de cas d'utilisation raffiné de la Release 1	40
3.4	Diagramme de classes de la Release 1	42
3.5	Sprint 1 : Authentification et Gestion des Utilisateurs	42
3.5.1	Objectifs du Sprint 1	42
3.5.2	Diagramme de cas d'utilisation global du Sprint 1	43
3.5.3	Analyse et Conception du Sprint 1	43
3.5.4	Réalisation du Sprint 1	47
3.6	Sprint 2 : Gestion du Stock, Catalogue et Navigation	48
3.6.1	Objectifs du Sprint 2	48
3.6.2	Diagramme de cas d'utilisation du Sprint 2	49
3.6.3	Analyse et Conception du Sprint 2	49
3.6.4	Réalisation du Sprint 2	53
3.7	Sprint 3 : Panier et Commandes	54
3.7.1	Objectifs du Sprint 3	54
3.7.2	Diagramme de cas d'utilisation du Sprint 3	54
3.7.3	Analyse et Conception du Sprint 3	55
3.7.4	Réalisation du Sprint 3	57
3.8	Conclusion	58

4 Release 2 : Fonctionnalités Avancées	59
4.1 Introduction	60
4.2 Backlog de la Release 2	60
4.3 Diagramme de cas d'utilisation de la Release 2	61
4.4 Diagramme de classes de la Release 2	62
4.5 Sprint 4 : Gestion des Tickets et Simulation de Performance	63
4.5.1 Objectifs du Sprint 4	63
4.5.2 Diagramme de cas d'utilisation du Sprint 4	64
4.5.3 Analyse et Conception du Sprint 4	64
4.5.4 Réalisation du Sprint 4	69
4.6 Sprint 5 : Intégration IA et Dashboard Analytique	71
4.6.1 Objectifs du Sprint 5	71
4.6.2 Diagramme de cas d'utilisation du Sprint 5	71
4.6.3 Analyse et Conception du Sprint 5	72
4.6.4 Réalisation du Sprint 5	76
4.7 Conclusion	77
Conclusion Générale	78
Webography	80

Liste des figures

1.1	Logo RedSys	4
1.2	Organigramme de RedSys	5
1.3	Processus SCRUM	10
1.4	Logo UML uml_logo	11
2.1	Diagramme de cas d'utilisation global	16
2.2	Diagramme de classes	17
2.3	Physical Architecture (N-Tier)	26
2.4	Architecture Logique (Three-Tier) — Next.js	26
2.5	Logo HTML5	27
2.6	Logo CSS3	28
2.7	Logo TypeScript	28
2.8	Logo Python	28
2.9	Illustration API REST	29
2.10	Logo FastAPI	29
2.11	Logo Next.js	30
2.12	Logo React	30
2.13	Logo Tailwind CSS	31
2.14	Logo shadcn/ui	31
2.15	Logo PostgreSQL	32
2.16	Logo Prisma ORM	32
2.17	Logo Docker	32
2.18	Logo Docker Compose	33
2.19	Logo GitHub Actions	33
2.20	Logo Vercel	34
2.21	Logo Visual Studio Code	34
2.22	Logo Postman	34
2.23	Logo pgAdmin	35
2.24	Logo Git	35
2.25	Logo GitHub	36
3.1	Diagramme de cas d'utilisation raffiné – Release 1	41

3.2	Diagramme de classes – Release 1	42
3.3	Diagramme de cas d'utilisation raffiné – Sprint 1	43
3.4	Diagramme de séquence – S'authentifier	45
3.5	Diagramme de séquence – Ajouter un Utilisateur	47
3.6	Sprint 1 – Interface de Connexion	48
3.7	Sprint 1 – Interface de Gestion des Utilisateurs	48
3.8	Diagramme de cas d'utilisation raffiné – Sprint 2	49
3.9	Diagramme de séquence – Gérer le Stock	51
3.10	Diagramme de séquence – Rechercher un Produit	53
3.11	Sprint 2 – Interface de Gestion du Stock	54
3.12	Sprint 2 – Interface Catalogue	54
3.13	Diagramme de cas d'utilisation – Sprint 3	55
3.14	Diagramme de séquence – Valider une Commande	57
3.15	Sprint 3 – Interface Panier et Commande	58
4.1	Diagramme de cas d'utilisation raffiné – Release 2	62
4.2	Diagramme de classes – Release 2	63
4.3	Diagramme de cas d'utilisation raffiné – Sprint 4	64
4.4	Diagramme de séquence – Soumettre un Ticket	67
4.5	Diagramme de séquence – Simuler une Charge	69
4.6	Sprint 4 – Interface de Gestion des Tickets de Support	70
4.7	Sprint 4 – Interface de Simulation de Performance	71
4.8	Diagramme de cas d'utilisation raffiné – Sprint 5	72
4.9	Diagramme de séquence – Obtenir une Recommandation	74
4.10	Diagramme de séquence – Consulter le Dashboard Analytique	76
4.11	Sprint 5 – Interface de Recommandation Intelligente	77
4.12	Sprint 5 – Tableau de Bord Analytique	77

Liste des tableaux

2.1	Backlog de produit - Plateforme Intelligente de Recommandation de l'Infrastructure IT	18
2.1	Product Backlog	22
2.2	Planning des Sprints	24
3.1	Backlog de la Release 1 (MVP)	40
3.2	Description textuelle – Cas d'utilisation «S'authentifie»	44
3.3	Description textuelle – Cas d'utilisation «Gestion des Utilisateurs»	46
3.4	Description textuelle – Cas d'utilisation «Gestion du Stock»	50
3.5	Description textuelle – Cas d'utilisation «Consulter le Catalogue»	52
3.6	Description textuelle – Cas d'utilisation «Gestion du Panier et Commandes»	56
4.1	Backlog de la Release 2 (Fonctionnalités Avancées)	61
4.2	Description textuelle – Cas d'utilisation «Gestion des Tickets de Support»	66
4.3	Description textuelle – Cas d'utilisation «Simulation de Performance»	68
4.4	Description textuelle – Cas d'utilisation «Recommandation Intelligente»	73
4.5	Description textuelle – Cas d'utilisation «Dashboard Analytique»	75

Liste des abréviations

Abbreviation	Description
API	Application Programming Interface
ACID	Atomicity, Consistency, Isolation, Durability
AI	Artificial Intelligence
CSS	Cascading Style Sheets
CRUD	Create, Read, Update, Delete
CI/CD	Continuous Integration / Continuous Deployment
DB	Database
DOM	Document Object Model
ELK	Elasticsearch, Logstash, Kibana
EV	Electric Vehicle
FCM	Firebase Cloud Messaging
FK	Foreign Key
GQL	GraphQL
GORM	Go Object Relational Mapper
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IDE	Integrated Development Environment
IT	Information Technology
JS	JavaScript
JSON	JavaScript Object Notation
JWT	JSON Web Token
MoSCoW	Must Have, Should Have, Could Have, Won't Have
MVC	Model View Controller
MVP	Minimum Viable Product
OCPP	Open Charge Point Protocol
OOP	Object Oriented Programming
ORM	Object Relational Mapper
OS	Operating System

Abbreviation	Description
PK	Primary Key
REST	Representational State Transfer
SaaS	Software as a Service
SDL	Schema Definition Language
SQL	Structured Query Language
SSL	Secure Sockets Layer
TLS	Transport Layer Security
TS	TypeScript
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
UX	User Experience
W3C	World Wide Web Consortium
WS	WebSocket

Introduction générale

Le marché des équipements informatiques reconditionnés connaît une croissance significative, portée par les impératifs de réduction des coûts et les enjeux environnementaux croissants. Face à cette dynamique, les entreprises peinent pourtant à sélectionner les infrastructures serveur les mieux adaptées à leurs besoins, faute d'outils d'analyse et de simulation dédiés. La complexité technique inhérente au choix d'un serveur reconditionné – tenant compte des paramètres CPU, RAM, stockage et charge applicative – nécessite une expertise avancée que la plupart des organisations ne possèdent pas en interne.

Dans ce contexte, **RedSys**, entreprise spécialisée dans les solutions IT et la commercialisation d'équipements informatiques reconditionnés, a identifié la nécessité de moderniser son offre à travers une plateforme web intelligente. L'approche actuelle, reposant sur une présentation statique des produits sans outil d'aide à la décision, ne permet pas aux clients entreprises d'évaluer objectivement la pertinence d'un serveur par rapport à leurs besoins réels. Cette lacune impacte directement la satisfaction client, la performance commerciale de RedSys et la valorisation de son catalogue de produits reconditionnés.

L'objectif de ce projet est de concevoir et développer une plateforme web intelligente composée de plusieurs modules complémentaires : un moteur de recommandation automatique basé sur l'intelligence artificielle pour orienter les clients vers les équipements les plus adaptés à leur profil, un simulateur de performance permettant de tester des scénarios de charge, un tableau de bord analytique décisionnel à destination de l'équipe RedSys, un espace de gestion du stock et des commandes, ainsi qu'un système de ticketing pour le support client. La plateforme intègre également une refonte complète du site vitrine de RedSys, offrant une expérience utilisateur moderne et cohérente.

La solution repose sur une architecture technique moderne et évolutive, combinant **Next.js** pour le frontend et le backend, **PostgreSQL** comme système de gestion de base de données relationnelle interfacé via **Prisma**, et une **API REST sécurisée** avec authentification **JWT**. Le moteur de recommandation s'appuie sur un **service IA externe** intégré à la plateforme, garantissant des recommandations précises et personnalisées.

Le processus de développement suit la méthodologie agile **Scrum**, organisée en sprints itératifs, assurant une livraison progressive de valeur et un alignement continu avec les besoins évolutifs de l'entreprise.

Ce rapport est structuré comme suit :

- **Chapitre 1** présente le contexte du projet, l'entreprise d'accueil RedSys, les limites du système existant et la solution proposée. Il introduit également la méthodologie Scrum et le

langage de modélisation UML adoptés tout au long du projet.

- **Chapitre 2** couvre l’analyse et la spécification des besoins. Il identifie les acteurs du système, définit les exigences fonctionnelles et non fonctionnelles, établit le product backlog, détaille le planning des sprints, présente l’architecture du système aux niveaux physique et logique, et justifie les choix techniques retenus.
- **Chapitre 3** détaille la conception et l’implémentation de la Release 1, couvrant les premiers sprints dédiés au site vitrine, au module de recommandation intelligente et au simulateur de performance.
- **Chapitre 4** se concentre sur la Release 2, présentant les fonctionnalités des sprints suivants : le tableau de bord analytique, la gestion du stock et des commandes, ainsi que le système de ticketing et support client.

Ce travail représente une étape concrète vers la transformation de RedSys en une plateforme intelligente d’aide à la décision, positionnant l’entreprise comme un acteur innovant dans la valorisation des infrastructures IT reconditionnées et posant les bases d’une solution scalable, fiable et orientée vers l’avenir.

CONTEXTE ET OBJECTIFS DU PROJET

Plan

1	Introduction	4
2	Présentation de l'organisme d'Accueil	4
3	Présentation du projet	5
4	Présentation de la méthodologie de gestion de projet	7
5	Présentation du langage de modélisation	10
6	Conclusion	11

1.1 Introduction

Ce chapitre présente le contexte du projet, l'entreprise d'accueil, la problématique identifiée, ainsi que la méthodologie adoptée pour le développement de la plateforme intelligente de recommandation et d'optimisation de l'infrastructure IT.

1.2 Présentation de l'organisme d'Accueil

1.2.1 Présentation de la Société RedSys

RedSys est une entreprise spécialisée dans les solutions IT, la gestion d'infrastructure et la commercialisation d'équipements informatiques reconditionnés. Reconnue pour son expertise dans le domaine des serveurs et composants informatiques, l'entreprise accompagne les organisations dans l'optimisation de leurs infrastructures en proposant des équipements de qualité à des coûts maîtrisés, contribuant ainsi à une démarche à la fois économique et responsable sur le plan environnemental.



FIGURE 1.1 : Logo RedSys

1.2.2 Mission et Vision de l'Entreprise

Mission : La mission de RedSys est d'être un partenaire de confiance pour les entreprises souhaitant optimiser leurs infrastructures informatiques à travers des équipements reconditionnés de haute qualité. En tant que spécialiste des solutions IT, l'entreprise s'engage à proposer des équipements fiables, des conseils techniques adaptés aux besoins de chaque client, et des services à forte valeur ajoutée, tout en respectant les valeurs d'excellence, d'innovation et de responsabilité environnementale.

Vision : RedSys aspire à devenir la référence incontournable dans le domaine des infrastructures IT reconditionnées, en se positionnant comme un acteur innovant capable d'accompagner les entreprises dans leurs décisions technologiques grâce à des outils intelligents d'aide à la décision. En intégrant l'intelligence artificielle dans son offre, RedSys entend créer une valeur durable pour ses clients, ses collaborateurs et l'ensemble de l'écosystème numérique.

1.2.3 Organigramme

La figure 1.2 illustre l'organigramme de RedSys

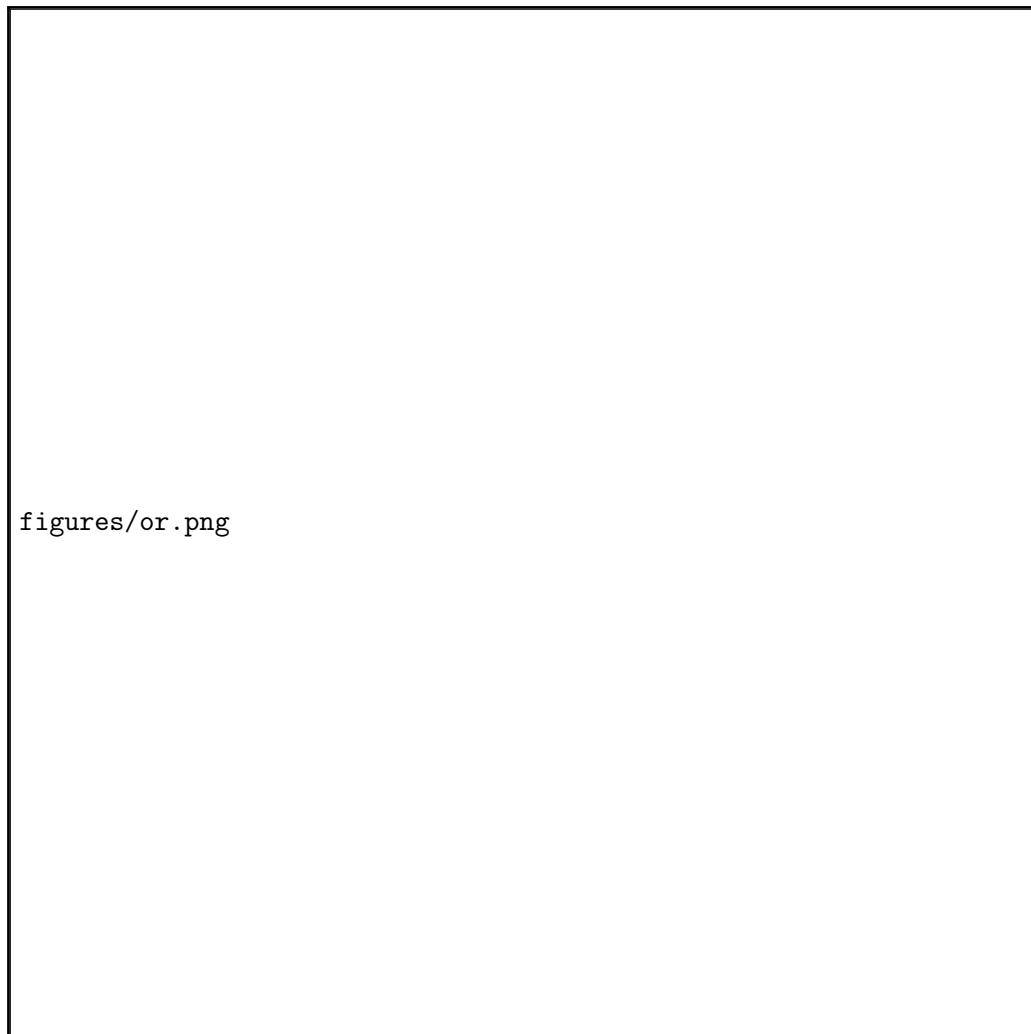


FIGURE 1.2 : Organigramme de RedSys

1.3 Présentation du projet

Pour mener à bien ce projet, une étude préalable a permis de décrire l'existant puis d'en déterminer les limites afin de produire une solution qui répondra au mieux aux besoins de notre client.

1.3.1 Etude de l'existant

RedSys s'appuie actuellement sur un site web statique pour présenter son catalogue d'équipements informatiques reconditionnés et attirer des clients entreprises. Si cette approche a permis d'assurer une présence en ligne, elle ne propose aucun outil d'aide à la décision permettant aux clients d'évaluer la pertinence d'un équipement par rapport à leurs besoins réels. Plusieurs plateformes intégrées existent sur le marché et tentent de répondre à des besoins similaires, notamment :

- **Renwtech** – une plateforme de vente d’équipements informatiques reconditionnés proposant des fiches techniques détaillées et un catalogue structuré par catégories de produits ;
- **ServerMonkey** – une plateforme e-commerce spécialisée dans les serveurs reconditionnés offrant des fonctionnalités de filtrage avancé par marque, configuration et cas d’usage ;
- **Corelink IT** – une solution de gestion d’infrastructure IT intégrant des outils de recommandation et de suivi du cycle de vie des équipements pour les entreprises.

Cependant, aucune de ces solutions ne répond pleinement aux besoins spécifiques de RedSys, notamment la combinaison d’un moteur de recommandation intelligent basé sur l’IA, d’un simulateur de performance intégré et d’un tableau de bord analytique décisionnel pour la gestion interne.

1.3.2 Critique de l’existant

L’analyse des solutions existantes révèle plusieurs limites récurrentes dans le contexte actuel :

- **Absence d’aide à la décision** : Les plateformes actuelles présentent les équipements sous forme de catalogues statiques sans proposer de recommandations personnalisées basées sur les besoins réels du client (nombre d’utilisateurs, type d’application, charge de travail) ;
- **Absence de simulation de performance** : Aucune solution existante ne permet au client de simuler des scénarios de charge afin d’évaluer le comportement d’un serveur avant l’achat, ce qui engendre des risques de mauvais dimensionnement ;
- **Scalabilité limitée** : Les outils de gestion de catalogue utilisés ne sont pas conçus pour évoluer vers des fonctionnalités analytiques avancées, sans possibilité d’intégrer des indicateurs de performance ;
- **Absence de traçabilité et de pilotage interne** : Les solutions existantes ne proposent pas de tableau de bord permettant à RedSys de suivre les tendances de consultation, les configurations les plus demandées, ou les produits à faible rotation, ce qui limite la capacité de prise de décision stratégique ;
- **Faible support client et suivi des demandes** : L’absence d’un système de ticketing intégré contraint les clients à recourir à des canaux de communication génériques (email, téléphone) pour soumettre des demandes de support, sans possibilité de suivi structuré de l’état de leurs tickets.

1.3.3 Solution proposée

Pour pallier les limites identifiées, RedSys propose le développement d’une plateforme web intelligente et intégrée, composée de cinq modules principaux :

- **Un module de recommandation intelligente** – un moteur s’appuyant sur un service

d'intelligence artificielle externe qui analyse le profil du client (nombre d'utilisateurs, type d'application, besoins en stockage) et recommande automatiquement les équipements les plus adaptés, avec une estimation du score de performance ;

- **Un module de simulation de performance** – un outil interactif permettant au client de simuler des scénarios de charge de travail et d'évaluer le taux d'utilisation CPU et RAM, le niveau de risque et la stabilité estimée du serveur dans des conditions d'utilisation réelles ;
- **Un tableau de bord analytique décisionnel** – une interface dédiée à l'équipe RedSys permettant de visualiser en temps réel les configurations les plus demandées, les produits à forte rotation, les taux de consultation et les indicateurs stratégiques de performance commerciale ;
- **Un module de gestion du stock et des commandes** – un espace d'administration permettant la gestion CRUD des équipements, la mise à jour des informations techniques, et côté client, la gestion du panier, la modification des quantités et la validation des demandes d'achat ;
- **Un module de gestion des tickets de support** – un système structuré permettant aux clients de soumettre des demandes de support, de suivre l'état de leurs tickets, et à l'administrateur RedSys de consulter, répondre et clôturer les demandes selon leur degré d'urgence.

Cette architecture remplace l'approche statique actuelle par une infrastructure fiable, sécurisée et évolutive. Elle garantit une traçabilité de bout en bout, réduit considérablement les erreurs de dimensionnement, et positionne RedSys pour une croissance future à travers des évolutions planifiées telles que l'analyse prédictive du cycle de vie des équipements, la détection d'anomalies par IA et un modèle SaaS multi-clients.

1.4 Présentation de la méthodologie de gestion de projet

L'agilité représente une approche dynamique et adaptative de la gestion de projet et du développement logiciel, mettant l'accent sur la flexibilité, la collaboration et la capacité à s'ajuster aux évolutions des exigences, plutôt que de se conformer strictement à un plan préétabli. Cette méthodologie s'articule autour de cycles courts, appelés itérations, favorisant ainsi une amélioration continue et une livraison rapide de produits fonctionnels et de qualité. Parmi les diverses méthodes Agiles, **Scrum** se distingue comme l'une des plus répandues et appréciées. Scrum constitue un cadre de travail (framework) à la fois simple et puissant, conçu pour permettre aux équipes de collaborer efficacement tout en répondant aux besoins spécifiques des clients. Grâce à ses principes et pratiques bien définis, Scrum facilite la

création de produits de haute qualité, tout en encourageant l'engagement et la responsabilité au sein des équipes.

1.4.1 Les rôles SCRUM

On a trois rôles principaux en SCRUM :

Product Owner (PO) Le Product Owner joue un rôle central dans la méthodologie Scrum en agissant comme le principal représentant des parties prenantes et des clients. Il est responsable de la priorisation des éléments du backlog produit, garantissant que l'équipe de développement se concentre sur les fonctionnalités les plus précieuses et pertinentes. En collaborant avec les utilisateurs finaux et en recueillant leurs retours, le PO s'assure que le produit final répond aux attentes et aux besoins du marché.

SCRUM Master Le Scrum Master est le facilitateur du processus Scrum, veillant à ce que les principes et les pratiques de la méthodologie soient respectés. Il joue un rôle de coach pour l'équipe, en les aidant à s'améliorer et à adopter une mentalité Agile. Le Scrum Master favorise un environnement de travail collaboratif et productif, permettant ainsi à chaque membre de l'équipe de donner le meilleur de lui-même

Équipe de Développement Composée de professionnels aux compétences variées, l'équipe de développement est chargée de concevoir, développer et livrer les fonctionnalités du produit à la fin de chaque sprint. Cette équipe autonome collabore étroitement pour transformer les exigences du backlog en résultats tangibles. En s'engageant à respecter les délais et à fournir un produit de haute qualité, l'équipe de développement joue un rôle crucial dans la réussite du projet, tout en s'adaptant aux retours et aux changements de priorités au fil des itérations.

1.4.2 Les événements Scrum

Dans le cadre de la méthodologie Scrum, plusieurs événements clés structurent le processus de développement, permettant ainsi une gestion efficace des projets et une collaboration optimale au sein des équipes.

Sprint Le sprint est une itération qui dure généralement entre 1 et 4 semaines, durant laquelle l'équipe de développement s'engage à livrer un incrément fonctionnel du produit. Chaque sprint commence par une planification minutieuse et se termine par une évaluation, garantissant ainsi que le produit évolue de manière continue et répond aux attentes des utilisateurs.[1]

Daily Scrum (Mêlée quotidienne) Cette réunion quotidienne, d’une durée de 15 minutes, permet à l’équipe de se synchroniser et de discuter des progrès réalisés, des obstacles rencontrés et des tâches à accomplir. Le Daily Scrum favorise la transparence et la communication au sein de l’équipe, renforçant ainsi la cohésion et l’engagement de chacun.[2]

Sprint Planning (Planification du sprint) Lors de cette réunion de planification, l’équipe détermine les tâches à réaliser durant le sprint en fonction des priorités établies dans le backlog produit. Le Sprint Planning permet de clarifier les objectifs du sprint et d’assurer que chaque membre de l’équipe comprend son rôle et ses responsabilités, ce qui contribue à une exécution fluide et efficace.[3]

Sprint Review (Revue de sprint) À la fin de chaque sprint, une présentation est faite aux parties prenantes pour montrer le travail accompli. Cette réunion, appelée Sprint Review, permet de recueillir des retours précieux et d’ajuster les priorités pour les sprints futurs. Elle favorise également l’engagement des parties prenantes et leur implication dans le processus de développement. [4]

Sprint Retrospective (Rétrospective de sprint) Cet événement, qui se déroule après la Sprint Review, est dédié à l’amélioration continue des pratiques de l’équipe. Lors de la Sprint Retrospective, les membres de l’équipe réfléchissent ensemble sur ce qui a bien fonctionné, ce qui pourrait être amélioré et les actions à mettre en place pour optimiser leur collaboration et leur efficacité lors des prochains sprints. Ce processus d’auto-évaluation est essentiel pour favoriser une culture d’apprentissage et d’adaptation au sein de l’équipe. [5]

1.4.3 Les artefacts de Scrum

Dans Scrum, les artefacts servent à matérialiser soit les efforts fournis, soit la valeur produite. Ils permettent d’assurer une transparence maximale et offrent des occasions régulières d’inspection et d’adaptation. Conçus pour clarifier les informations essentielles, ils garantissent une compréhension commune au sein de toute l’équipe.

Backlog Produit Un backlog produit est une liste hiérarchisée de tâches destinées à l’équipe de développement. Il est créé à partir de la feuille de route produit et de ses exigences. Les éléments les plus importants figurent en tête du backlog produit. Ainsi, l’équipe sait ce qu’elle doit livrer en priorité. L’équipe de développement ne suit pas le backlog au rythme du Product Owner et celui-ci n’impose pas de tâches à l’équipe de développement. Cette dernière récupère les tâches à réaliser dans le backlog de produit en fonction de ses capacités, soit en continu (Kanban), soit par itération (Scrum).[6]

Backlog Sprint Le Backlog Sprint est un sous-ensemble du Backlog Produit, sélectionné et affiné pour être livré durant un Sprint donné. Il regroupe les éléments que l'équipe de développement s'engage à transformer en un incrément fonctionnel et potentiellement publiable d'ici la fin du Sprint. En plus des éléments choisis, le Backlog Sprint inclut un plan détaillé sur la manière dont le travail sera accompli, souvent représenté par des tâches techniques ou des sous-éléments définis par les développeurs eux-mêmes. Le Backlog Sprint constitue ainsi la prévision de l'équipe sur ce qu'elle pense pouvoir réaliser pendant l'itération en cours. Il reste cependant un artefact dynamique : l'équipe peut l'ajuster en cours de Sprint pour mieux atteindre l'objectif fixé. L'ensemble du travail intégré dans le Backlog Sprint doit, à l'issue du Sprint, satisfaire aux standards de qualité établis et être conforme à la définition de « Terminé » adoptée par l'équipe Scrum. [7]

Incrément L'incrément est constitué des éléments du Backlog Produit qui ont été jugés « Finis » durant le sprint, ainsi que de la valeur cumulative des incréments livrés lors des sprints précédents. À l'issue d'un Sprint, le nouvel incrément doit être « Fini », ce qui signifie qu'il doit être dans un état prêt à être publié et conforme à la définition de « Fini » (Definition of Done) établie par l'équipe de développement. Un incrément représente une portion de travail réalisée, qui soutient l'approche empirique et peut être vérifiée à la fin du Sprint. Il constitue une avancée vers une vision ou un objectif, et doit être dans un état prêt à être publié, indépendamment de la décision du Product Owner de le rendre public ou non.[8]

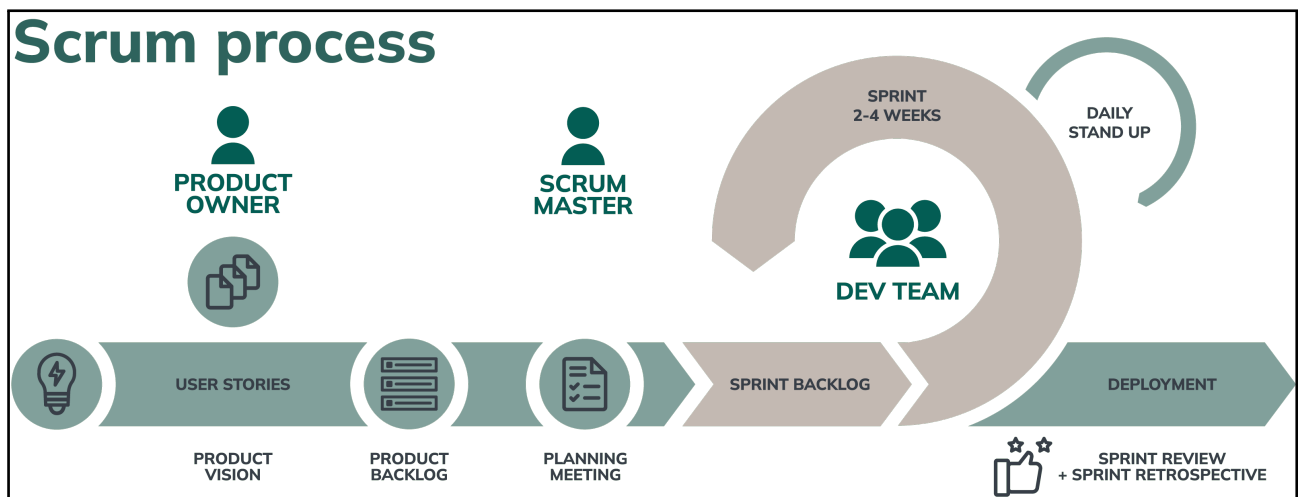


FIGURE 1.3 : Processus SCRUM

1.5 Présentation du langage de modélisation

Ce projet repose sur les principes de la programmation orientée objet, ce qui nécessite l'utilisation d'un langage de modélisation adapté à ce paradigme. Pour cela, le meilleur choisi est l'UML (Unified Modeling Language), qui est reconnu pour sa capacité à représenter de manière visuelle et sémantique

la structure et le comportement de systèmes logiciels complexes.

UML est un langage de modélisation. La version actuelle, **UML 2.5**, propose **14 types de diagrammes** dont sept structurels et sept comportementaux.

UML n'étant pas une méthode, l'utilisation des diagrammes est laissée à l'appréciation de chacun. Le diagramme de classes est généralement considéré comme l'élément central d'UML. Des méthodes, telles que le processus unifié proposé par les créateurs originels de UML, utilisent plus systématiquement l'ensemble des diagrammes et axent l'analyse sur les cas d'utilisation (« use case ») pour développer par itérations successives un modèle d'analyse, un modèle de conception, et d'autres modèles. D'autres approches se contentent de modéliser seulement partiellement un système, par exemple certaines parties critiques qui sont difficiles à déduire du code. [9]



FIGURE 1.4 : Logo UML `uml_logo`

1.6 Conclusion

Dans ce premier chapitre, nous avons exposé le contexte du projet, identifié les problèmes rencontrés, et proposé des critiques ainsi que des solutions potentielles. Nous avons également justifié le choix de la méthodologie de travail adoptée, en mettant en avant les éléments indicateurs qu'elle offre, ainsi que notre langage de modélisation. Dans le chapitre suivant, nous procéderons à l'analyse et à la spécification des besoins du projet.

ANALYSE ET SPÉCIFICATION DES BESOINS

Plan

1	Introduction	13
2	Spécification des besoins	13
3	Diagramme de cas d'utilisation global	15
4	Diagramme de Classes	16
5	Structure et découpage du projet	17
6	Architecture de l'Application	25
7	Choix Techniques et Environnement de Développement	27
8	Conclusion	36

2.1 Introduction

Dans ce chapitre, nous présentons une analyse des aspects fondamentaux de notre projet. En commençant par la précision des principaux acteurs et des fonctionnalités qui leur sont associées, passant par la définition des besoins fonctionnels et non fonctionnels, le diagramme de cas d'utilisation global et le backlog de produit. nous détaillerons l'architecture générale de l'application et l'environnement de travail.

2.2 Spécification des besoins

Dans cette section on va identifier les acteurs de l'application, les besoins fonctionnels et non fonctionnels.

2.2.1 Identification des acteurs

Un acteur est représenté par toute personne, équipe ou entité qui a un intérêt, une influence ou un impact sur le projet ou le produit développé.

- **Visiteur** : un utilisateur non inscrit ou non connecté qui peut accéder aux informations publiques de l'application ,sans pouvoir interagir avec les fonctionnalités réservées aux clients.
- **Client entreprise** : Représente toute organisation souhaitant acquérir ou évaluer des équipements informatiques reconditionnés. Il interagit avec la plateforme pour obtenir des recommandations personnalisées, simuler des scénarios de performance, consulter le catalogue, gérer son panier et soumettre des demandes de support.
- **Administrateur RedSys** : C'est le modérateur de l'application , chargée de gérer, superviser et maintenir la plateforme et la base de données, en assurant son bon fonctionnement et sa sécurité.
- **Employé RedSys** : Un collaborateur interne de RedSys disposant d'un accès limité à certaines fonctionnalités de gestion, notamment la mise à jour de son profil, le traitement des tickets assignées par l'administrateur.

2.2.2 Spécification des besoins fonctionnels

Nous précisons les besoins fonctionnels par acteur.

Visiteur

- Consulter librement le catalogue des équipements (serveurs, stockage, réseau) sans authentification ;
- Rechercher et filtrer les produits selon plusieurs critères (marque, prix, capacité, performance,

type d'équipement) ;

- Consulter les détails d'un produit (caractéristiques techniques, état du produit, prix, images) ;
- Configurer un serveur en sélectionnant différents composants (RAM, stockage, processeur) selon ses besoins ;
- Ajouter des produits au panier et modifier les quantités ;
- Supprimer des produits du panier ;
- Consulter le récapitulatif du panier (prix total, liste des produits sélectionnés) ;

Client Entreprise

- S'authentifier de manière sécurisée à la plateforme ;
- Saisir ses besoins (nombre d'utilisateurs, type d'application, besoin en stockage) et obtenir une recommandation automatique de serveurs adaptés ;
- Consulter les détails des équipements recommandés (serveur, RAM, stockage conseillé, score de performance estimé) ;
- Simuler des scénarios de charge de travail et visualiser les indicateurs de performance (taux CPU, taux RAM, niveau de risque, stabilité) ;
- Parcourir le catalogue d'équipements, ajouter des produits au panier, modifier les quantités et valider une demande de commande ;
- Soumettre une demande de support (panne, réclamation) et suivre l'état de ses tickets.

Administrateur RedSys

- S'authentifier et accéder au tableau de bord d'administration ;
- Gérer les comptes utilisateurs : créer, modifier et désactiver les comptes employés ;
- Gérer le catalogue des équipements : ajout, modification, suppression et mise à jour des informations techniques ;
- Visualiser les indicateurs analytiques : types de serveurs les plus demandés, configurations les plus recommandées, taux de consultation, produits à forte et faible rotation ;
- Consulter et affecter les tickets de support clients aux employés ;
- Traiter les demandes de commandes clients et mettre à jour leur statut ;
- Gérer les rôles, les permissions d'accès et assurer la sécurité de la plateforme.

Employé

- S'authentifier à la plateforme avec les droits qui lui sont attribués ;
- Consulter et répondre aux tickets de support assignés selon ses permissions ;

2.2.3 Spécification des besoins non fonctionnels

Les besoins non fonctionnels désignent les critères qui définissent la qualité et les contraintes d'un système. Ils agissent indirectement sur les résultats du système et sur la productivité des utilisateurs.

- **Ergonomie** : La plateforme doit proposer une interface intuitive, claire et agréable, nécessitant un minimum de formation aussi bien pour les clients entreprises que pour les équipes internes de RedSys.
- **Responsivité** : L'application web doit offrir une expérience optimale sur une large gamme d'appareils et de résolutions d'écran, en adaptant dynamiquement sa mise en page.
- **Performance** : Le système doit afficher les recommandations et résultats de simulation en moins de 3 secondes ; l'interface doit rester fluide et réactive même lors de traitements intensifs côté serveur.
- **Disponibilité** : La plateforme doit maintenir un taux de disponibilité de 99,9% afin de garantir un accès continu et ininterrompu aux clients et aux équipes RedSys.
- **Scalabilité** : L'architecture doit supporter une croissance du catalogue d'équipements et une augmentation du nombre d'utilisateurs simultanés sans dégradation des performances.
- **Sécurité et Confidentialité** : Le système doit garantir la protection complète des données personnelles et commerciales. Tous les flux de données doivent être chiffrés via TLS, l'authentification assurée par JWT, et les accès strictement limités aux utilisateurs autorisés selon leurs rôles. Les mots de passe ne doivent jamais être stockés ou transmis en clair.
- **Traçabilité** : Toutes les actions critiques (modifications du catalogue, traitement des commandes, réponses aux tickets) doivent être journalisées et auditables à tout moment.
- **Maintenabilité et Évolutivité** : Le système doit être conçu avec une architecture modulaire et extensible, capable d'intégrer de futures évolutions telles que l'analyse prédictive du cycle de vie des équipements, la détection d'anomalies par IA et un modèle SaaS multi-clients.

2.3 Diagramme de cas d'utilisation global

Les diagrammes des cas d'utilisation identifient les fonctionnalités fournies par le système (cas d'utilisation), les utilisateurs qui interagissent avec le système (acteurs), et l'interaction entre ces derniers. Les principaux objectifs des diagrammes des cas d'utilisation sont :

- Fournir une vue de haut-niveau de ce que fait le système.
- Identifier les utilisateurs (acteurs) du système.

La Figure 2.1 présente le diagramme de cas d'utilisation global :

Remarque : toutes les fonctionnalités nécessitent une authentification, sauf celles du visiteur.



FIGURE 2.1 : Diagramme de cas d'utilisation global

2.4 Diagramme de Classes

Le diagramme de classes est un type de diagramme UML utilisé pour modéliser la structure statique d'un système. Il représente les classes du système, leurs attributs, leurs méthodes (opérations), ainsi que les relations entre ces classes.

La Figure 2.2 montre notre diagramme de classes.

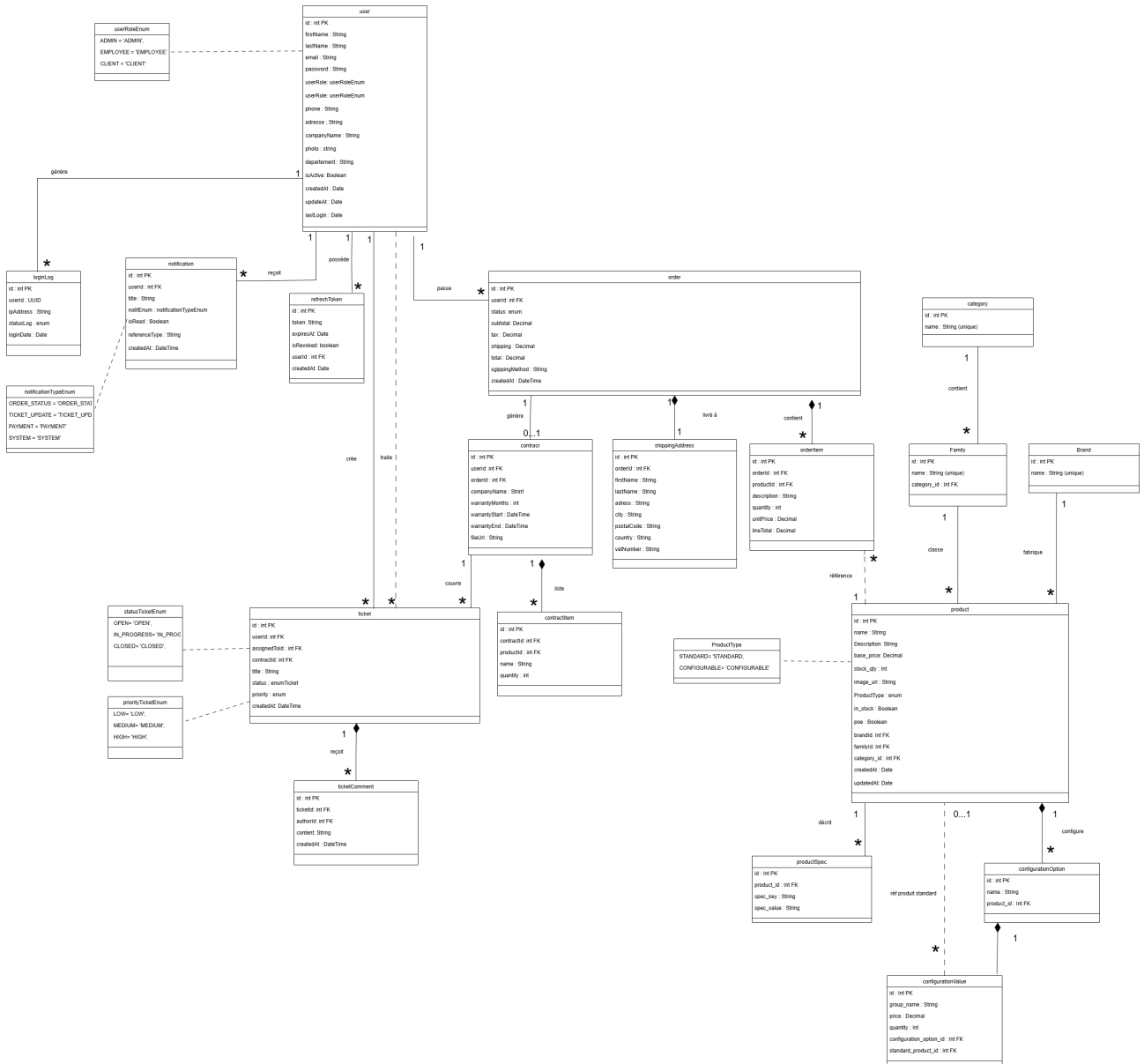


FIGURE 2.2 : Diagramme de classes

2.5 Structure et découpage du projet

2.5.1 Identification de l'équipe SCRUM

L'équipe Scrum de ce projet est composée de :

- **Product Owner** : Yassine Khammassi
- **Scrum Master** : Mohamed Mehdi Kaouech
- **Developer** : Mohamed Aziz Merai

2.5.2 Backlog de produit

Le tableau 2.1 représente le backlog de produit de notre application .

Légende

Priorité MoSCoW :

Must - Must have	Fonctionnalité essentielle, requise pour le MVP
Should - Should have	Fonctionnalité importante mais non critique
Could - Could have	Fonctionnalité souhaitable mais non nécessaire
Won't - Won't have	Fonctionnalité non incluse dans cette version

Complexité :

1	Simple - Effort minimal, implémentation directe
2	Modérée - Effort modéré, complexité moyenne
3	Complexe - Effort significatif, implémentation complexe
5	Très complexe - Effort important, haute complexité

TABLEAU 2.1 : Backlog de produit - Plateforme Intelligente de Recommandation de l'Infrastructure IT

ID	Feature	US ID	User Story	Priority	Complexi
1	Authentification	1.1	En tant qu'utilisateur, je veux me connecter de manière sécurisée à la plateforme.	Must	1
		1.2	En tant qu'utilisateur, je veux réinitialiser mon mot de passe via un processus sécurisé.	Should	2
		1.3	En tant qu'administrateur, je veux que le contrôle d'accès basé sur les rôles soit appliqué sur toute la plateforme.	Must	2
2	Gestion des Utilisateurs	2.1	En tant qu'administrateur, je veux créer, modifier et désactiver les comptes employés.	Must	2

Suite à la page suivante...

ID	Feature	US ID	User Story	Priority	Complexi
		2.2	En tant qu'administrateur, je veux gérer les rôles et les permissions d'accès de chaque utilisateur.	Must	2
		2.3	En tant qu'administrateur, je veux consulter les statistiques d'utilisation et les journaux d'audit de la plateforme.	Should	3
3	Gestion du Stock	3.1	En tant qu'administrateur, je veux gérer le catalogue CRUD des équipements (ajout, modification, suppression).	Must	2
		3.2	En tant qu'administrateur, je veux mettre à jour les informations techniques des équipements.	Must	1
		3.3	En tant qu'administrateur, je veux consulter et traiter les commandes soumises par les clients.	Must	2
		3.4	En tant qu'administrateur, je veux recevoir une alerte lorsqu'un produit est en rupture de stock.	Should	2
4	Catalogue et Navigation	4.1	En tant que visiteur, je veux consulter librement le catalogue des équipements sans authentification.	Must	1
		4.2	En tant que visiteur, je veux rechercher et filtrer les produits selon plusieurs critères (marque, prix, type).	Must	2
		4.3	En tant que visiteur, je veux consulter les détails d'un produit (caractéristiques techniques, état, prix, images).	Must	1

Suite à la page suivante...

ID	Feature	US ID	User Story	Priority	Complexi
		4.4	En tant que visiteur, je veux configurer un serveur en sélectionnant différents composants (RAM, stockage, processeur) selon mes besoins.	Should	3
		4.5	En tant que client, je veux parcourir le catalogue et ajouter des équipements au panier depuis les résultats de recommandation.	Must	1
5	Panier et Commandes	5.1	En tant que visiteur, je veux ajouter des produits au panier et modifier les quantités.	Must	1
		5.2	En tant que visiteur, je veux supprimer des produits de mon panier.	Must	1
		5.3	En tant que visiteur, je veux consulter le récapitulatif de mon panier (prix total, liste des produits sélectionnés).	Must	1
		5.4	En tant que client, je veux valider ma demande de commande et suivre son état après soumission.	Must	2
6	Gestion des Tickets de Support	6.1	En tant que client, je veux soumettre une demande de support (panne, réclamation) via un formulaire dédié.	Must	2
		6.2	En tant que client, je veux suivre l'état de mes tickets (en cours, traité, fermé).	Must	1
		6.3	En tant qu'administrateur, je veux consulter les tickets clients et les affecter aux employés concernés.	Must	2
		6.4	En tant qu'employé, je veux consulter et répondre aux tickets de support qui me sont assignés.	Must	2

Suite à la page suivante...

ID	Feature	US ID	User Story	Priority	Complexi
7	Simulation de Performance	7.1	En tant que client, je veux simuler des scénarios de charge de travail sur un serveur pour évaluer ses capacités.	Must	3
		7.2	En tant que client, je veux visualiser les indicateurs de performance (taux CPU, taux RAM, niveau de risque, stabilité) après simulation.	Must	2
8	Intégration IA	8.1	En tant que client, je veux saisir mes besoins (nombre d'utilisateurs, type d'application, stockage) pour obtenir une recommandation automatique de serveurs adaptés.	Must	3
		8.2	En tant que client, je veux consulter les détails des équipements recommandés (serveur, RAM, stockage conseillé, score de performance estimé).	Must	2
		8.3	En tant que client, je veux que le système utilise un service IA externe pour affiner les recommandations selon mon profil.	Must	3
		8.4	En tant que client, je veux interagir avec un chatbot pour poser des questions sur les produits et obtenir des réponses instantanées.	Should	3
		8.5	En tant que client, je veux recevoir des suggestions personnalisées de produits basées sur mon historique d'achat et mon profil utilisateur ;		
9	Dashboard Analytique	9.1	En tant qu'administrateur, je veux visualiser les types de serveurs les plus demandés et les configurations les plus recommandées.	Must	3

Suite à la page suivante...

ID	Feature	US ID	User Story	Priority	Complexi
		9.2	En tant qu'administrateur, je veux filtrer, trier et exporter les données analytiques.	Should	2

TABLEAU 2.1 : Product Backlog

2.5.3 Planification des Sprints

Une fois que le Backlog est préparé, l'étape suivante consiste à définir la durée du sprint et à répartir les éléments du backlog de produit sur un nombre de sprints.

Le tableau 2.2 montre la planification des sprints .

Release	Sprint	Durée (Jours)	Fonctionnalités
Release 1	Sprint 1	19	Authentification + Gestion des Utilisateurs (6 user stories – 12 pts) • Connexion sécurisée à la plateforme • Réinitialisation du mot de passe • Contrôle d'accès basé sur les rôles • Création, modification et désactivation des comptes employés • Gestion des rôles et permissions d'accès • Consultation des statistiques et journaux d'audit

Suite à la page suivante...

Release	Sprint	Durée (Jours)	Fonctionnalités
	Sprint 2	21	Gestion du Stock + Catalogue et Navigation (9 user stories – 15 pts) • Gestion CRUD du catalogue d'équipements • Mise à jour des informations techniques des équipements • Consultation et traitement des commandes clients • Alerte de rupture de stock • Consultation libre du catalogue sans authentification • Recherche et filtrage multicritères (marque, prix, type) • Fiche produit détaillée (caractéristiques, état, prix, images) • Configurateur de serveur (RAM, stockage, processeur) • Ajout au panier depuis les résultats de recommandation
	Sprint 3	9	Panier et Commandes (4 user stories – 5 pts) • Ajout de produits au panier et modification des quantités • Suppression de produits du panier • Récapitulatif du panier (prix total, liste des produits) • Validation de commande et suivi de son état

Suite à la page suivante...

Release	Sprint	Durée (Jours)	Fonctionnalités
Release 2	Sprint 4	19	Gestion des Tickets + Simulation de Performance (6 user stories – 12 pts) • Soumission d’une demande de support par le client • Suivi de l’état des tickets par le client • Affectation des tickets aux employés par l’administrateur • Consultation et réponse aux tickets par l’employé • Simulation de scénarios de charge de travail sur un serveur • Visualisation des indicateurs de performance (CPU, RAM, risque, stabilité)
	Sprint 5	21	Intégration IA + Dashboard Analytique (7 user stories – 18 pts) • Saisie des besoins pour une recommandation automatique de serveurs • Consultation des détails des équipements recommandés • Recommandation affinée via un service IA externe • Chatbot pour questions sur les produits • Suggestions personnalisées basées sur l’historique d’achat • Visualisation des serveurs les plus demandés et configurations recommandées • Filtrage, tri et export des données analytiques
Résumé : 32 User Stories – 62 Points – 89 Jours			

TABLEAU 2.2 : Planning des Sprints

2.6 Architecture de l'Application

Dans cette section, nous présentons l'architecture de l'application. Nous nous intéressons à l'architecture physique et logique de la solution.

2.6.1 Architecture Physique

L'architecture physique du système suit un modèle **N-Tiers**, qui sépare le système en couches distinctes, chacune dotée d'une responsabilité bien définie. Cette séparation garantit la scalabilité, la maintenabilité et des frontières claires entre les différents composants de la plateforme.

L'architecture physique est organisée en trois couches :

- **Couche Présentation :**

Cette couche englobe l'ensemble des interfaces côté client :

- **Application web Next.js** accessible par les visiteurs, les clients entreprises, les employés et les administrateurs RedSys depuis n'importe quel navigateur.

- **Couche Application :**

Cette couche héberge la logique métier centrale du système. Elle est constituée de deux composants principaux :

- **Serveur Next.js (API Routes)** : point d'entrée unique pour toutes les requêtes client, intégrant frontend et backend au sein du même projet. Il gère l'authentification JWT, le contrôle d'accès basé sur les rôles et l'orchestration de la logique métier ;
- **Service IA externe** : service indépendant dédié au moteur de recommandation intelligent, interrogé par le serveur Next.js via des appels API sécurisés pour générer les recommandations de serveurs et les scores de performance.

- **Couche Données :**

Cette couche est responsable de la persistance des données :

- **PostgreSQL** : base de données relationnelle principale stockant l'ensemble des données de la plateforme (utilisateurs, équipements, commandes, tickets, recommandations) ;
- **Prisma ORM** : interface de communication entre le serveur Next.js et PostgreSQL, offrant un accès typé, des migrations automatisées et une modélisation déclarative du schéma de données.

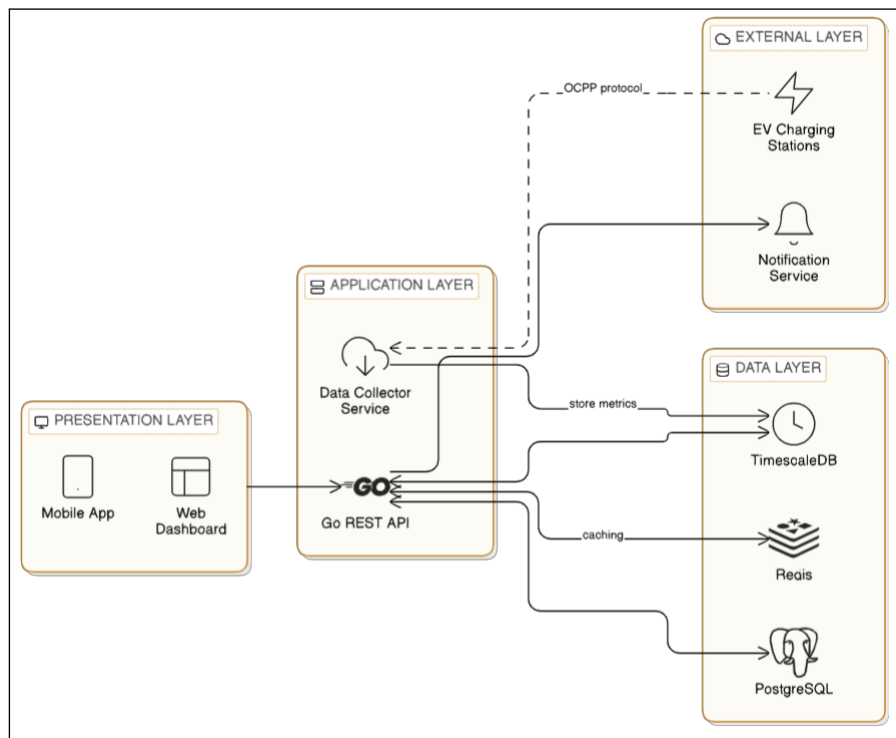


FIGURE 2.3 : Physical Architecture (N-Tier)

2.6.2 Architecture Logique

L'architecture logique du système suit une architecture **Three-Tier** (trois niveaux), naturellement supportée par le framework Next.js, qui unifie frontend et backend au sein d'un même projet tout en maintenant une séparation claire des responsabilités :

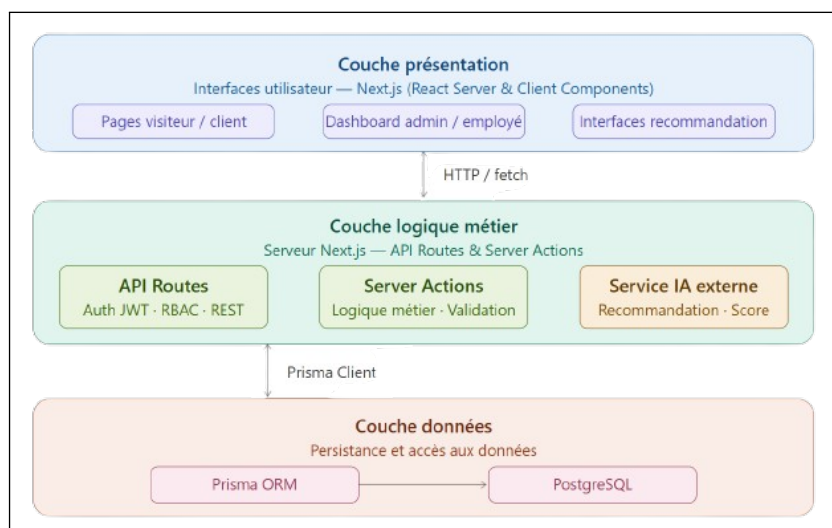


FIGURE 2.4 : Architecture Logique (Three-Tier) — Next.js

Le pattern trois niveaux est appliqué comme suit :

- **Couche Présentation** : Pages et composants React rendus côté client, responsables de l'interface utilisateur et de la capture des interactions.
- **Couche Logique Métier** : API Routes et Server Actions Next.js exécutés côté serveur,

gérant l'authentification JWT, les règles métier et la communication avec le service IA externe.

- **Couche Données** : Prisma ORM interfacé avec PostgreSQL, assurant la persistance et l'accès structuré aux données.

2.7 Choix Techniques et Environnement de Développement

Cette section présente les technologies, frameworks, langages et outils sélectionnés pour le développement du système.

2.7.1 Langages de Programmation

2.7.1.1 HTML

HTML (HyperText Markup Language) est le langage de balisage standard pour la création de pages web [1]. Il décrit la structure du contenu web à l'aide d'un système d'éléments représentés par des balises. HTML5, la norme actuelle maintenue par le W3C, a introduit des éléments sémantiques, le support multimédia et des fonctionnalités d'accessibilité améliorées qui constituent l'épine dorsale structurelle des applications web modernes.



FIGURE 2.5 : Logo HTML5

La figure 2.5 présente le logo de HTML5.

2.7.1.2 CSS

CSS (Cascading Style Sheets) est le langage utilisé pour décrire la présentation d'un document écrit en HTML [2]. Il contrôle la mise en page, les couleurs, la typographie, l'espacement et la responsivité sur différentes tailles d'écran. CSS3, la version actuelle, a introduit des fonctionnalités avancées telles que flexbox, grid layout, les animations et les media queries, permettant la création d'interfaces web entièrement responsives et visuellement cohérentes.



FIGURE 2.6 : Logo CSS3

La figure 2.6 présente le logo de CSS3.

2.7.1.3 TypeScript

TypeScript est un langage de programmation open-source développé par Microsoft qui étend JavaScript en ajoutant un typage statique optionnel [3]. Il se compile en JavaScript pur et est entièrement compatible avec les bases de code JavaScript existantes. Le système de types de TypeScript permet la détection précoce des erreurs à la compilation, améliore la lisibilité du code et offre un puissant support IDE via l'autocomplétion et la documentation en ligne. Dans ce projet, TypeScript est utilisé à la fois côté frontend et côté backend au sein du projet Next.js.



FIGURE 2.7 : Logo TypeScript

La figure 2.7 présente le logo de TypeScript.

2.7.1.4 Python

Python est un langage de programmation interprété, généraliste et open-source, reconnu pour sa lisibilité et sa richesse en bibliothèques scientifiques **python_lang**. Il est aujourd'hui le langage de référence pour le développement de solutions d'intelligence artificielle et de machine learning, grâce à des bibliothèques telles que scikit-learn, pandas et FastAPI. Dans ce projet, Python est utilisé pour le développement du service IA externe dédié au moteur de recommandation intelligente de serveurs.



FIGURE 2.8 : Logo Python

La figure 2.8 présente le logo de Python.

2.7.2 Couche API

2.7.2.1 API REST

Une API REST (Representational State Transfer) est une interface de programmation d'application qui respecte les contraintes architecturales REST **rest_arch**. Elle expose des ressources via des endpoints HTTP standard (GET, POST, PUT, DELETE) et repose sur des échanges de données au format JSON. Dans ce projet, l'API REST est implémentée au sein des API Routes de Next.js, servant de point de communication central entre le frontend et la base de données, tout en appliquant les règles d'authentification JWT et de contrôle d'accès basé sur les rôles.



FIGURE 2.9 : Illustration API REST

La figure 2.9 présente une illustration de l'API REST.

2.7.2.2 FastAPI

FastAPI est un framework web moderne et performant pour la création d'APIs avec Python **fastapi_framework**. Il est basé sur les annotations de types Python et génère automatiquement une documentation interactive via Swagger UI. FastAPI est utilisé dans ce projet pour exposer le service IA externe sous forme d'une API REST indépendante, consommée par le serveur Next.js pour les fonctionnalités de recommandation et de simulation de performance.

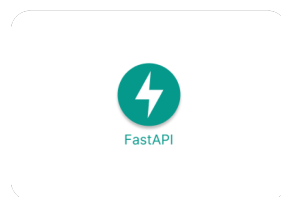


FIGURE 2.10 : Logo FastAPI

La figure 2.10 présente le logo de FastAPI.

2.7.3 Frameworks et Bibliothèques

2.7.3.1 Next.js

Next.js est un framework React open-source développé par Vercel **nextjs_framework**. Il offre le rendu côté serveur (SSR), la génération de pages statiques (SSG), le routage basé sur les

fichiers et les API Routes intégrées, permettant de développer à la fois le frontend et le backend au sein d'un seul et même projet. Next.js est aujourd'hui l'un des frameworks les plus adoptés pour la création d'applications web modernes, performantes et optimisées pour le référencement. Dans ce projet, Next.js constitue le socle unique de la plateforme RedSys, intégrant l'ensemble de la logique frontend et backend.



FIGURE 2.11 : Logo Next.js

La figure 2.11 présente le logo de Next.js.

2.7.3.2 React

React est une bibliothèque JavaScript open-source développée et maintenue par Meta pour la construction d'interfaces utilisateur [4]. Elle a introduit une architecture basée sur des composants et un DOM virtuel qui met à jour efficacement uniquement les parties de l'interface ayant changé. Le flux de données unidirectionnel de React et son riche écosystème de bibliothèques complémentaires en font le framework frontend le plus largement adopté pour la création d'applications web complexes et interactives. Next.js s'appuie sur React comme fondation pour le rendu des interfaces côté serveur et côté client.

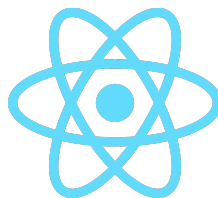


FIGURE 2.12 : Logo React

La figure 2.12 présente le logo de React.

2.7.3.3 Tailwind CSS

Tailwind CSS est un framework CSS utilitaire open-source **tailwind_css**. Contrairement aux frameworks traditionnels comme Bootstrap, Tailwind ne fournit pas de composants prédéfinis, mais un ensemble de classes utilitaires de bas niveau permettant de construire des interfaces personnalisées directement dans le balisage HTML. Son approche *utility-first* permet un développement rapide et une personnalisation totale de l'interface, tout en maintenant un fichier CSS minimal en production grâce à son mécanisme de purge automatique.



FIGURE 2.13 : Logo Tailwind CSS

La figure 2.13 présente le logo de Tailwind CSS.

2.7.3.4 shadcn/ui

shadcn/ui est une collection de composants d'interface utilisateur réutilisables, accessibles et hautement personnalisables, construits sur Radix UI et stylisés avec Tailwind CSS **shadcn_ui**. Contrairement aux bibliothèques de composants traditionnelles, shadcn/ui ne s'installe pas comme une dépendance npm — les composants sont copiés directement dans le projet et peuvent être modifiés librement. Cette approche garantit un contrôle total sur le code, une accessibilité native et une cohérence visuelle parfaite avec le design system de la plateforme RedSys.



FIGURE 2.14 : Logo shadcn/ui

La figure 2.14 présente le logo de shadcn/ui.

2.7.4 Base de Données et ORM

2.7.4.1 PostgreSQL

PostgreSQL est un système de gestion de base de données objet-relationnel puissant et open-source, avec plus de 35 ans de développement actif [5]. Il est entièrement conforme ACID et supporte des fonctionnalités avancées telles que les colonnes JSON, la recherche plein texte, les clés étrangères, les triggers et les procédures stockées. PostgreSQL est largement reconnu comme l'une des bases de données open-source les plus fiables et riches en fonctionnalités disponibles, ce qui en fait le choix naturel pour stocker l'ensemble des données de la plateforme RedSys.

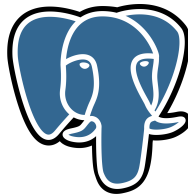


FIGURE 2.15 : Logo PostgreSQL

La figure 2.15 présente le logo de PostgreSQL.

2.7.4.2 Prisma ORM

Prisma est un ORM (Object-Relational Mapper) open-source de nouvelle génération pour Node.js et TypeScript **prisma_orm**. Il génère un client de base de données entièrement typé à partir d'un schéma déclaratif, offrant une autocomplétion complète et une détection d'erreurs à la compilation. Prisma gère également les migrations de base de données de manière automatisée grâce à sa CLI intégrée, et s'interface nativement avec PostgreSQL. Son schéma déclaratif constitue la source de vérité unique pour la modélisation des entités du domaine dans ce projet.



FIGURE 2.16 : Logo Prisma ORM

La figure 2.16 présente le logo de Prisma ORM.

2.7.5 DevOps et Déploiement

2.7.5.1 Docker

Docker est une plateforme open-source pour le développement, la livraison et l'exécution d'applications dans des conteneurs [6]. Les conteneurs encapsulent une application et toutes ses dépendances dans une unité portable unique, garantissant un comportement cohérent entre les environnements de développement, de test et de production. Dans ce projet, Docker est utilisé pour conteneuriser l'application Next.js, le service IA Python et la base de données PostgreSQL.

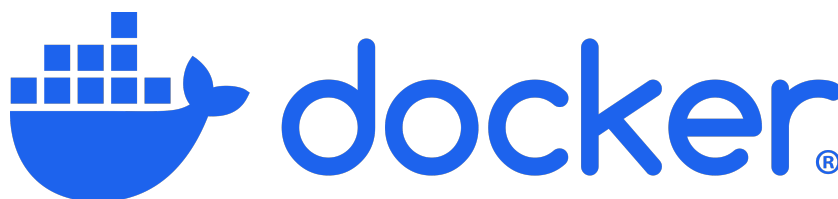


FIGURE 2.17 : Logo Docker

La figure 2.17 présente le logo de Docker.

2.7.5.2 Docker Compose

Docker Compose est un outil permettant de définir et d'orchestrer des applications multi-conteneurs à l'aide d'un seul fichier de configuration YAML [6]. Il permet de démarrer l'ensemble des services du projet (Next.js, service IA Python, PostgreSQL) avec une seule commande, en gérant automatiquement les dépendances entre conteneurs, les volumes persistants et les réseaux internes. Docker Compose simplifie considérablement la mise en place de l'environnement de développement local et de déploiement.

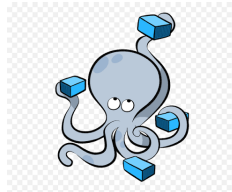


FIGURE 2.18 : Logo Docker Compose

La figure 2.18 présente le logo de Docker Compose.

2.7.5.3 GitHub Actions

GitHub Actions est une plateforme d'intégration continue et de déploiement continu (CI/CD) intégrée nativement à GitHub [7]. Elle permet d'automatiser les workflows de build, de test et de déploiement directement depuis le dépôt de code. Dans ce projet, GitHub Actions est utilisé pour automatiser la vérification du code TypeScript, l'exécution des tests et le déploiement de l'application à chaque push sur la branche principale, garantissant ainsi une livraison continue et fiable.

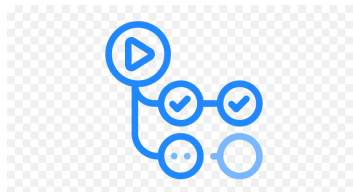


FIGURE 2.19 : Logo GitHub Actions

La figure 2.19 présente le logo de GitHub Actions.

2.7.5.4 Vercel

Vercel est la plateforme de déploiement cloud développée par les créateurs de Next.js **vercel_platform**. Elle offre un déploiement automatique à chaque push Git, un réseau de distribution de contenu (CDN) mondial, la gestion des variables d'environnement et un support natif de toutes les fonctionnalités Next.js (SSR, SSG, API Routes, Server Actions). Vercel constitue la solution de déploiement de référence pour les projets Next.js grâce à son intégration transparente et ses performances optimisées.



FIGURE 2.20 : Logo Vercel

La figure 2.20 présente le logo de Vercel.

2.7.6 Outils de Développement

2.7.6.1 Visual Studio Code

Visual Studio Code est un éditeur de code gratuit et open-source développé par Microsoft [8]. Il supporte une large gamme de langages de programmation et offre un riche marketplace d'extensions couvrant TypeScript, React, Next.js, Prisma, Tailwind CSS et Python. Son architecture légère combinée à des fonctionnalités puissantes telles qu'IntelliSense, le terminal intégré, le débogueur et le support Git natif en font l'environnement de développement standard pour l'ensemble de ce projet.



FIGURE 2.21 : Logo Visual Studio Code

La figure 2.21 présente le logo de Visual Studio Code.

2.7.6.2 Postman

Postman est une plateforme collaborative pour le développement et le test d'APIs [9]. Elle fournit une interface graphique intuitive permettant de concevoir, tester et documenter les endpoints REST. Dans ce projet, Postman est utilisé pour tester les API Routes de Next.js et les endpoints du service IA Python, en permettant d'envoyer des requêtes HTTP, d'inspecter les réponses et de valider le comportement des différents modules avant leur intégration.



FIGURE 2.22 : Logo Postman

La figure 2.22 présente le logo de Postman.

2.7.6.3 pgAdmin

pgAdmin est l'outil d'administration et de gestion open-source de référence pour PostgreSQL `pgadmin_tool`. Il offre une interface graphique web permettant de visualiser la structure de la base de données, d'exécuter des requêtes SQL, de gérer les tables, les index et les relations, ainsi que de surveiller les performances du serveur PostgreSQL. Dans ce projet, pgAdmin est utilisé pour administrer la base de données de développement, vérifier les migrations Prisma et effectuer des requêtes d'analyse directement sur les données.

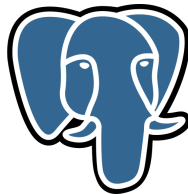


FIGURE 2.23 : Logo pgAdmin

La figure 2.23 présente le logo de pgAdmin.

2.7.6.4 Git

Git est un système de contrôle de version distribué, gratuit et open-source, conçu pour gérer des projets de toute taille avec rapidité et efficacité [10]. Il permet à plusieurs développeurs de travailler simultanément sur la même base de code grâce au mécanisme de branches et de fusions, et maintient un historique complet de toutes les modifications apportées au projet.



FIGURE 2.24 : Logo Git

La figure 2.24 présente le logo de Git.

2.7.6.5 GitHub

GitHub est une plateforme d'hébergement web pour les dépôts Git [7]. Elle fournit des outils pour la revue de code, le suivi des problèmes, la gestion de projet et l'intégration continue via GitHub Actions. GitHub sert de dépôt distant central pour ce projet et constitue le point d'entrée du pipeline CI/CD mis en place pour automatiser les tests et le déploiement de la plateforme RedSys.



FIGURE 2.25 : Logo GitHub

La figure 2.25 présente le logo de GitHub.

2.8 Conclusion

Le troisième chapitre décrit l'étude conceptuelle qui est divisée en deux parties, la première englobe la conception technique, alors que la seconde est destinée à la description des choix techniques et de l'environnement de travail. La description de la première release fera l'objet du chapitre suivant. Dans le deuxième chapitre, nous avons identifié les acteurs, les besoins fonctionnels et non fonctionnels de notre application, le Backlog du produit et la planification des sprints dans la 1ère partie alors que la seconde est destinée à la description des choix techniques et de l'environnement de travail. Dans les chapitres qui suivent, nous expliquerons de manière détaillée chaque release

RELEASE 1 : MVP

Plan

1	Introduction	38
2	Backlog de la Release 1	38
3	Diagramme de cas d'utilisation raffiné de la Release 1	40
4	Diagramme de classes de la Release 1	42
5	Sprint 1 : Authentification et Gestion des Utilisateurs	42
6	Sprint 2 : Gestion du Stock, Catalogue et Navigation	48
7	Sprint 3 : Panier et Commandes	54
8	Conclusion	58

3.1 Introduction

Dans ce chapitre, nous présenterons les différentes étapes de la première release : le module d'authentification et de gestion des utilisateurs, le module de gestion du stock, catalogue et navigation, ainsi que le module de panier et de gestion des commandes.

3.2 Backlog de la Release 1

Le tableau 3.1 décrit le backlog de produit du release 1.

ID	Fonctionnalité	US ID	User Story	Priorité	Complexité
1	Authentification + Gestion des Utilisateurs	1.1	En tant qu'utilisateur, je veux me connecter de manière sécurisée à la plateforme.	Must	1
		1.2	En tant qu'utilisateur, je veux réinitialiser mon mot de passe via un processus sécurisé.	Should	2
		1.3	En tant qu'administrateur, je veux que le contrôle d'accès basé sur les rôles soit appliqué sur toute la plateforme.	Must	2
		1.4	En tant qu'administrateur, je veux créer, modifier et désactiver les comptes employés.	Must	2
		1.5	En tant qu'administrateur, je veux gérer les rôles et les permissions d'accès de chaque utilisateur.	Must	2
		1.6	En tant qu'administrateur, je veux consulter les statistiques d'utilisation et les journaux d'audit.	Should	3
	Gestion du Stock	2.1	En tant qu'administrateur, je veux gérer le catalogue CRUD des équipements (ajout, modification, suppression).	Must	2

2 + Catalogue et
Navigation

Suite à la page suivante...

ID	Fonctionnalité	US ID	User Story	Priorité	Complexité
		2.2	En tant qu'administrateur, je veux mettre à jour les informations techniques des équipements.	Must	1
		2.3	En tant qu'administrateur, je veux consulter et traiter les commandes soumises par les clients.	Must	2
		2.4	En tant qu'administrateur, je veux recevoir une alerte lorsqu'un produit est en rupture de stock.	Should	2
		2.5	En tant que visiteur, je veux consulter librement le catalogue des équipements sans authentification.	Must	1
		2.6	En tant que visiteur, je veux rechercher et filtrer les produits selon plusieurs critères (marque, prix, type).	Must	2
		2.7	En tant que visiteur, je veux consulter les détails d'un produit (caractéristiques techniques, état, prix, images).	Must	1
		2.8	En tant que visiteur, je veux configurer un serveur en sélectionnant différents composants (RAM, stockage, processeur) selon mes besoins.	Should	3
		2.9	En tant que client, je veux ajouter des équipements au panier depuis les résultats de recherche du catalogue.	Must	1
3	Panier et Commandes	3.1	En tant que visiteur, je veux ajouter des produits au panier et modifier les quantités.	Must	1
		3.2	En tant que visiteur, je veux supprimer des produits de mon panier.	Must	1

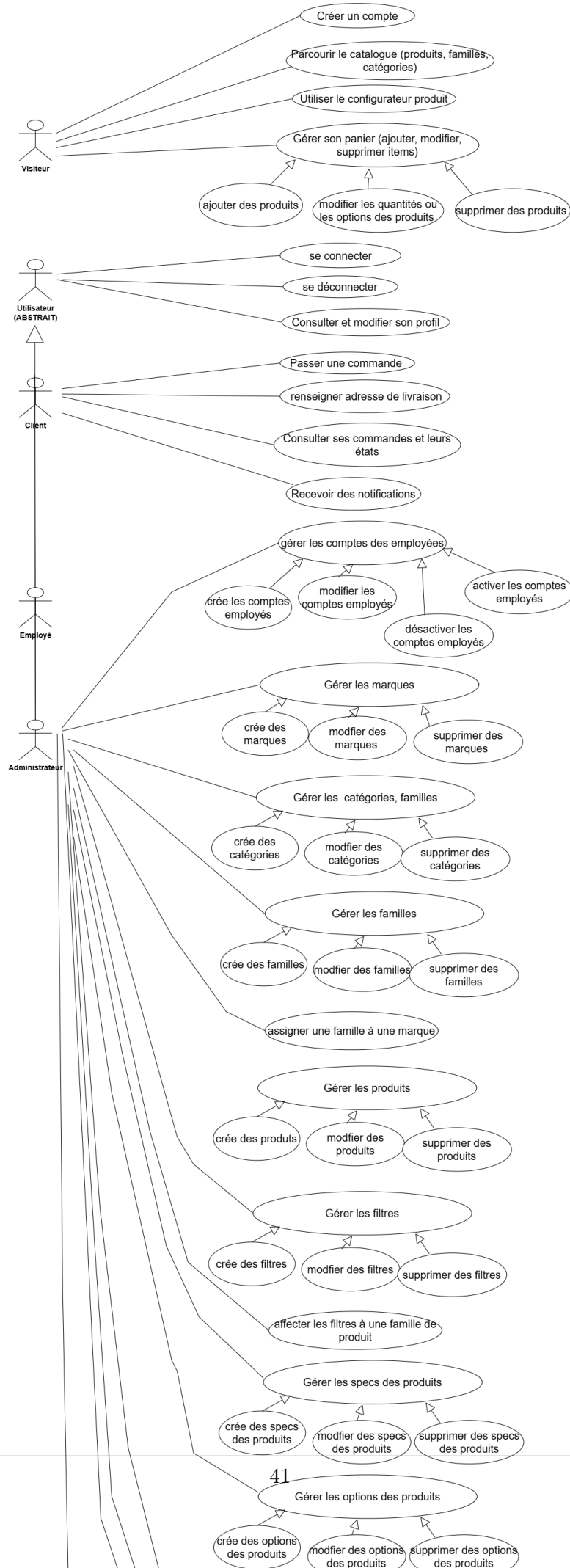
Suite à la page suivante...

ID	Fonctionnalité	US ID	User Story	Priorité	Complexité
		3.3	En tant que visiteur, je veux consulter le récapitulatif de mon panier (prix total, liste des produits sélectionnés).	Must	1
		3.4	En tant que client, je veux valider ma demande de commande et suivre son état après soumission.	Must	2

TABLEAU 3.1 : Backlog de la Release 1 (MVP)

3.3 Diagramme de cas d'utilisation raffiné de la Release 1

La figure 3.1 présente le diagramme de cas d'utilisation raffiné de la Release 1, couvrant les trois modules principaux : l'authentification et la gestion des utilisateurs, la gestion du stock couplée au catalogue et à la navigation, ainsi que le panier et la gestion des commandes.



3.4 Diagramme de classes de la Release 1

La figure 3.2 présente le diagramme de classes de la Release 1, illustrant les principales entités impliquées : Utilisateur, Rôle, Équipement, Stock, Panier et Commande, ainsi que leurs attributs, méthodes et relations.

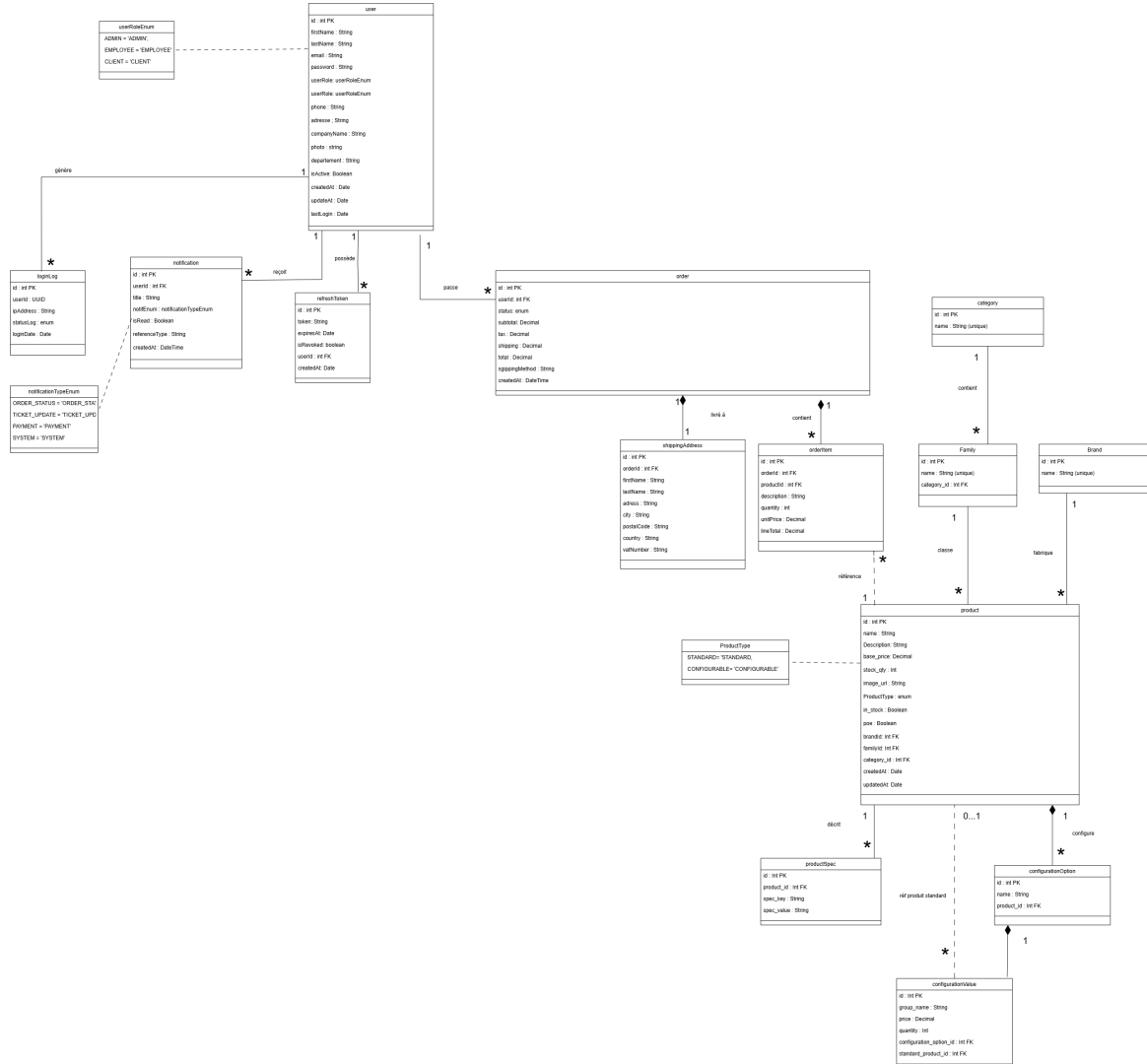


FIGURE 3.2 : Diagramme de classes – Release 1

3.5 Sprint 1 : Authentification et Gestion des Utilisateurs

3.5.1 Objectifs du Sprint 1

L'objectif du Sprint 1 est d'établir le socle de sécurité de la plateforme. Ce sprint livre le système d'authentification permettant aux visiteurs, clients, employés et administrateurs de se connecter de manière sécurisée, ainsi que le module de gestion des utilisateurs permettant à l'administrateur de créer, modifier et désactiver les comptes employés et de gérer les permissions d'accès.

3.5.2 Diagramme de cas d'utilisation global du Sprint 1

La figure 3.3 présente le diagramme de cas d'utilisation global du Sprint 1, détaillant les interactions entre l'Administrateur et les autres acteurs avec les fonctionnalités d'authentification et de gestion des utilisateurs.



FIGURE 3.3 : Diagramme de cas d'utilisation raffiné – Sprint 1

3.5.3 Analyse et Conception du Sprint 1

Pour que les cas d'utilisation soient plus compréhensibles, nous avons mis en place le raffinement de chacun dans le but de livrer une description sur les différents scénarios qui peuvent se présenter.

3.5.3.1 Cas d'utilisation «authentification»

• Description textuelle

Le tableau 3.2 décrit la description du scénario "authentification".

Cas d'utilisation	S'authentifier
Acteur	Administrateur, Client Entreprise, Employé
Objectif	Permettre aux utilisateurs du système de se connecter de manière sécurisée et d'accéder à leur espace de travail selon leur rôle assigné.
Précondition	L'utilisateur doit disposer d'un compte valide avec un email et un mot de passe enregistrés dans le système.
Post-condition	Après authentification réussie, l'utilisateur est redirigé vers l'interface correspondant à son rôle.
Scénario nominal	1. L'utilisateur accède à la page de connexion de la plateforme. 2. Le système affiche le formulaire d'authentification. 3. L'utilisateur saisit ses identifiants (email et mot de passe). 4. Le système valide les identifiants via l'API Route Next.js. 5. Le système génère un token JWT et établit la session. 6. L'utilisateur est redirigé vers son tableau de bord selon son rôle.
Scénario alternatif	A1 Les identifiants saisis sont incorrects. A2 Le système affiche un message d'erreur sur l'écran. A3 Le système redirige l'utilisateur vers la page de connexion et reprend à l'étape 2 du scénario nominal jusqu'à ce que l'authentification réussisse ou que la session soit abandonnée.

TABLEAU 3.2 : Description textuelle – Cas d'utilisation «S'authentifier»

• Diagramme de séquence

La figure 3.4 présente le diagramme de séquence du cas d'utilisation S'authentifier, illustrant les interactions entre l'utilisateur, l'interface Next.js, l'API Route d'authentification, Prisma ORM et la base de données PostgreSQL.

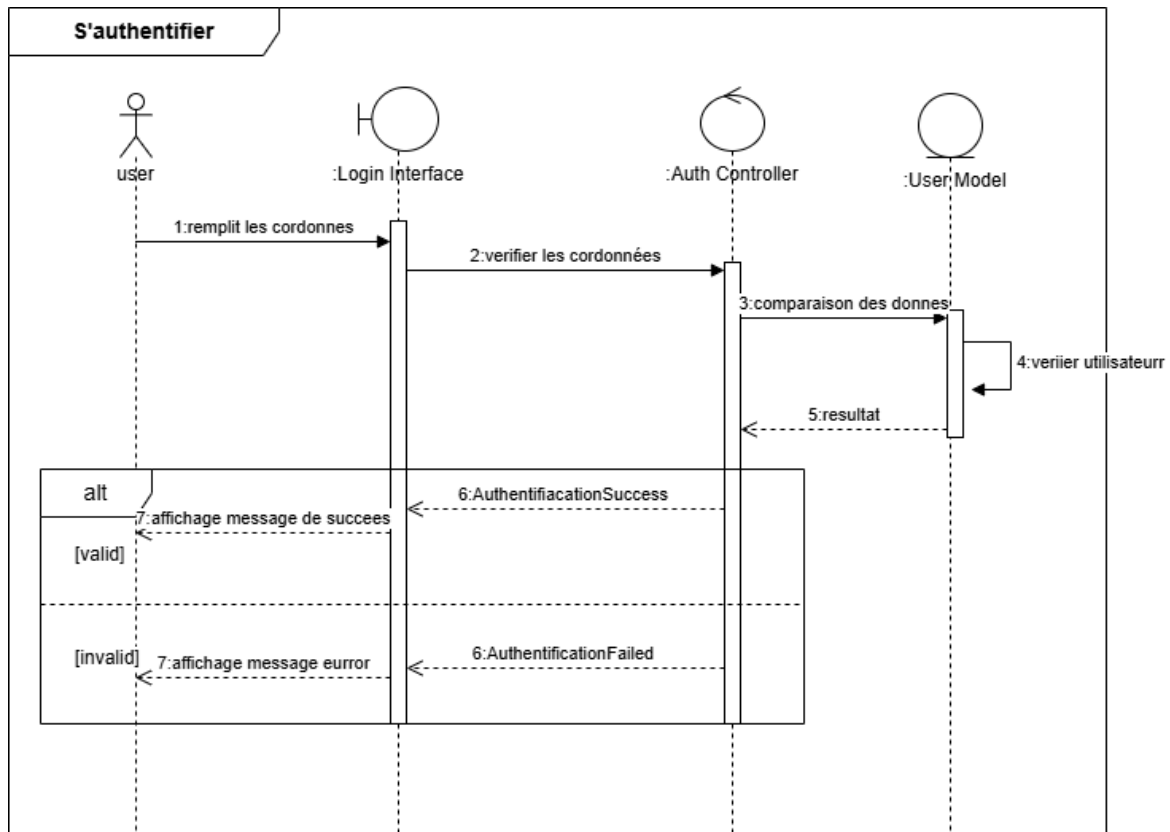


FIGURE 3.4 : Diagramme de séquence – S'authentifier

3.5.3.2 Cas d'utilisation «Gestion des Utilisateurs»

- Description textuelle

Cas d'utilisation	Gestion des Utilisateurs
Acteur	Administrateur RedSys
Objectif	Permettre à l'administrateur de créer, modifier, désactiver tous les comptes employés de la plateforme.
Précondition	L'administrateur doit être authentifié et disposer des privilèges administratifs complets.
Post-condition	Le compte employé est créé, mis à jour ou désactivé avec succès, et les modifications sont persistées dans la base de données.
Scénario nominal	1. L'administrateur accède à l'interface de gestion des utilisateurs. 2. Le système affiche la liste des comptes existants. 3. L'administrateur sélectionne l'action souhaitée (créer, modifier ou désactiver). 4. L'administrateur renseigne les informations requises. 5. Le système valide les données saisies via Prisma ORM. 6. Le système sauvegarde les modifications et confirme l'opération.
Scénario alternatif	A1 Les données fournies sont invalides ou incomplètes. A2 Le système affiche un message d'erreur de validation. A3 Le système retourne à l'étape 4 du scénario nominal jusqu'à ce que des données valides soient soumises.

TABLEAU 3.3 : Description textuelle – Cas d'utilisation «Gestion des Utilisateurs»

• Diagramme de séquence

La figure 3.5 présente le diagramme de séquence du cas d'utilisation Ajouter un Utilisateur, illustrant les interactions entre l'administrateur, l'interface Next.js, l'API Route, Prisma ORM et la

base de données PostgreSQL lors du processus de création de compte.

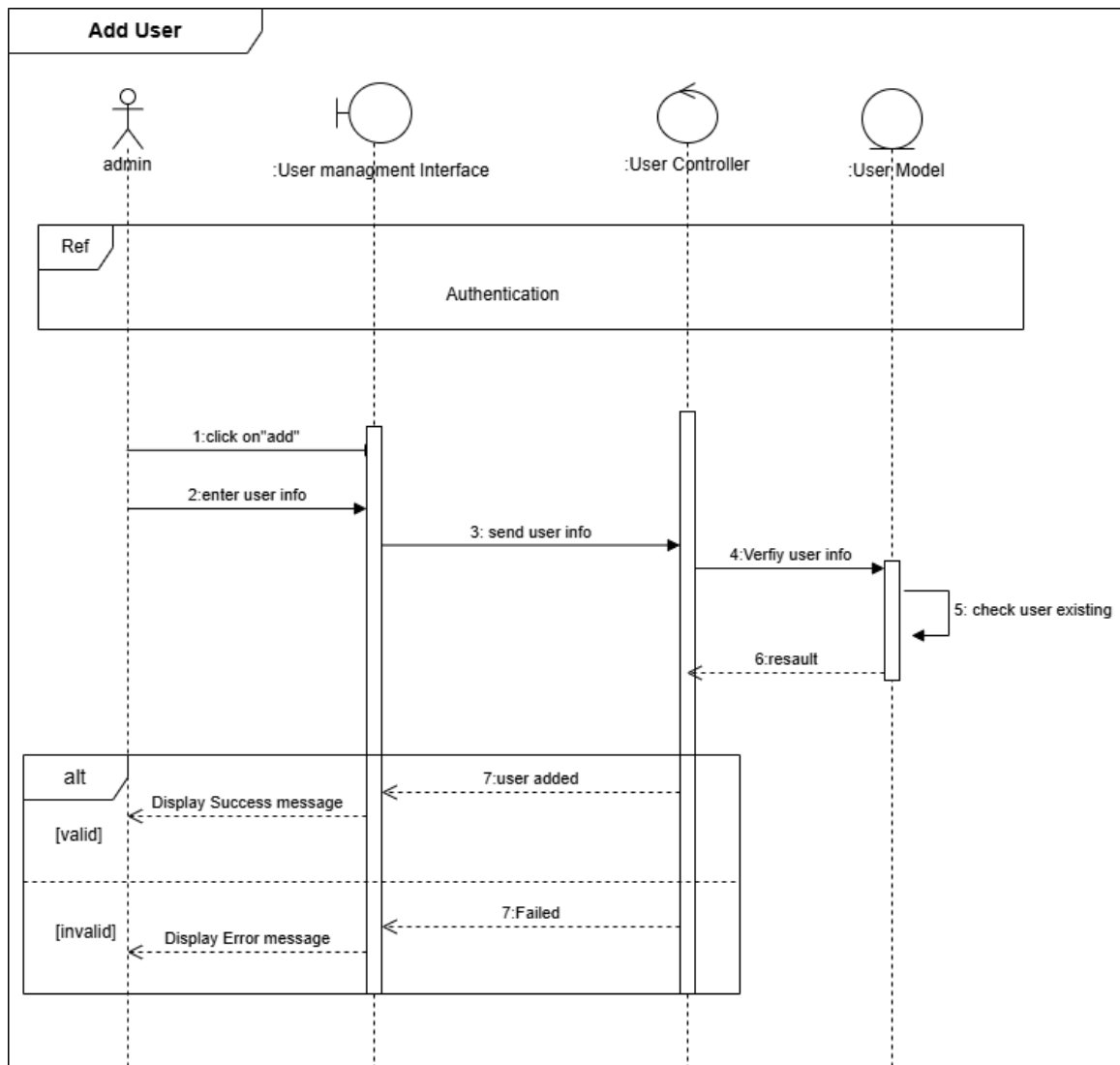


FIGURE 3.5 : Diagramme de séquence – Ajouter un Utilisateur

3.5.4 Réalisation du Sprint 1

Les figures 3.6 et 3.7 présentent les interfaces implémentées pour le Sprint 1, couvrant l'écran de connexion et le tableau de bord de gestion des utilisateurs

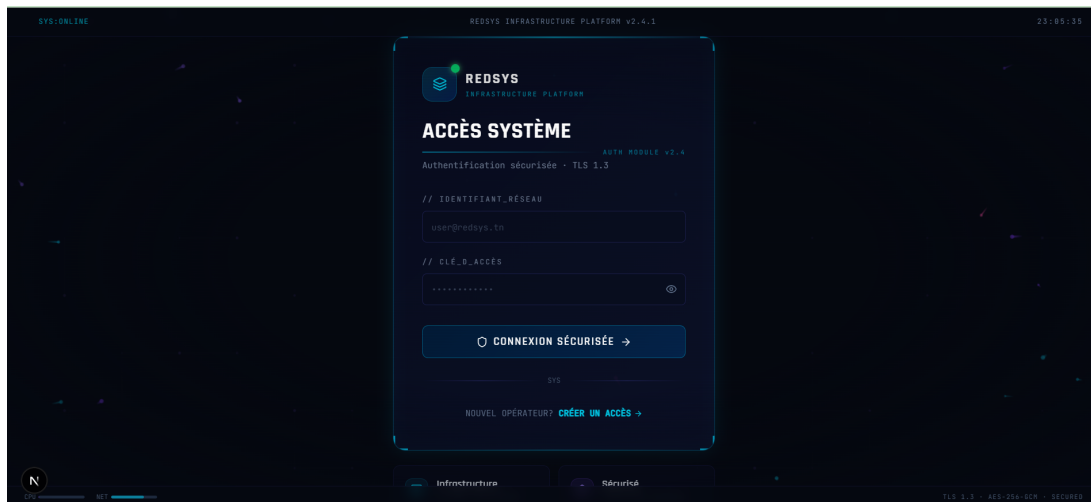


FIGURE 3.6 : Sprint 1 – Interface de Connexion

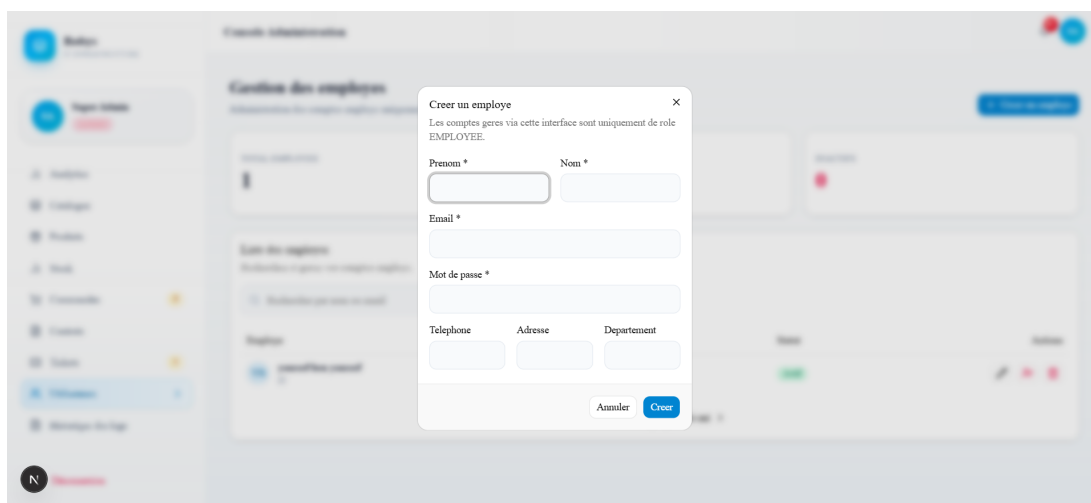


FIGURE 3.7 : Sprint 1 – Interface de Gestion des Utilisateurs

3.6 Sprint 2 : Gestion du Stock, Catalogue et Navigation

3.6.1 Objectifs du Sprint 2

L'objectif du Sprint 2 est de livrer le module de gestion du stock côté administrateur, couplé au module catalogue et navigation côté visiteur. L'administrateur pourra gérer le cycle de vie complet des équipements (ajout, modification, suppression), traiter les commandes soumises par les clients et recevoir des alertes automatiques en cas de rupture de stock. En parallèle, les visiteurs pourront consulter librement le catalogue des équipements reconditionnés, rechercher et filtrer les produits, consulter les fiches détaillées et configurer un serveur selon leurs besoins.

3.6.2 Diagramme de cas d'utilisation du Sprint 2

La figure 3.8 présente le diagramme de cas d'utilisation du Sprint 2, détaillant les interactions entre l'Administrateur et le Visiteur avec les fonctionnalités de gestion du stock, du catalogue et de la navigation.

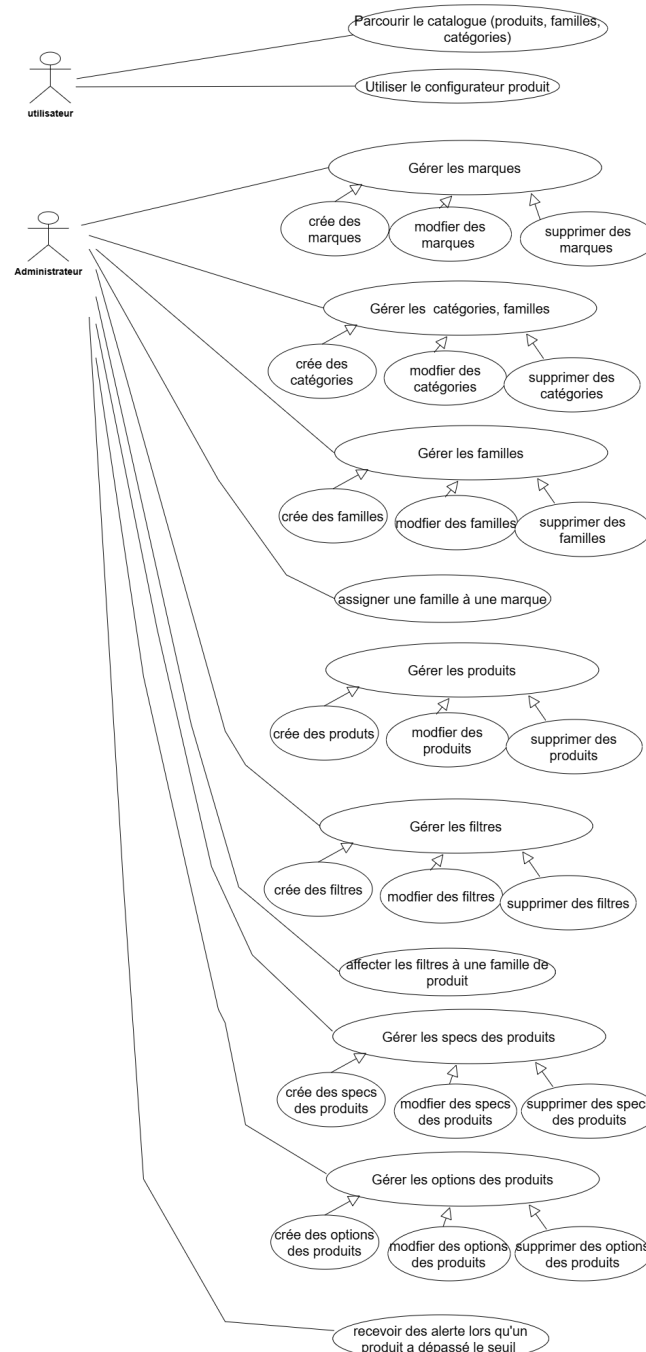


FIGURE 3.8 : Diagramme de cas d'utilisation raffiné – Sprint 2

3.6.3 Analyse et Conception du Sprint 2

3.6.3.1 Cas d'utilisation «Gestion du Stock»

- **Description textuelle** Le tableau 3.5 montre la description du scénario "Gestion du Stock".

Cas d'utilisation	Gestion du Stock
Acteur	Administrateur RedSys
Objectif	Permettre à l'administrateur de gérer le catalogue d'équipements (ajout, modification, suppression), de traiter les commandes soumises par les clients et d'être alerté automatiquement en cas de rupture de stock.
Précondition	L'administrateur doit être authentifié et disposer des privilèges de gestion du catalogue.
Post-condition	Le catalogue est mis à jour, la commande est traitée et le niveau de stock est ajusté en conséquence.
Scénario nominal	1. L'administrateur accède à l'interface de gestion du stock. 2. Le système affiche la liste des équipements avec leur quantité disponible. 3. L'administrateur ajoute, modifie ou supprime un équipement du catalogue. 4. Le système enregistre la modification via une transaction Prisma atomique. 5. Lorsqu'une commande est validée, le système décrémente automatiquement le stock de l'équipement concerné. 6. Si le stock d'un équipement passe sous le seuil défini, le système génère une notification de rupture destinée à l'administrateur.
Scénario alternatif	A1 La quantité commandée dépasse le stock disponible. A2 Le système bloque la validation de la commande côté serveur. A3 Le système informe l'administrateur et le client de l'indisponibilité de la quantité demandée.

TABLEAU 3.4 : Description textuelle – Cas d'utilisation «Gestion du Stock»

• Diagramme de séquence

La figure 3.9 présente le diagramme de séquence du cas d'utilisation Gérer le Stock, illustrant les interactions entre l'administrateur, l'interface Next.js, l'API Route, Prisma ORM et PostgreSQL lors de la décrémentation atomique du stock et de la génération d'alertes.

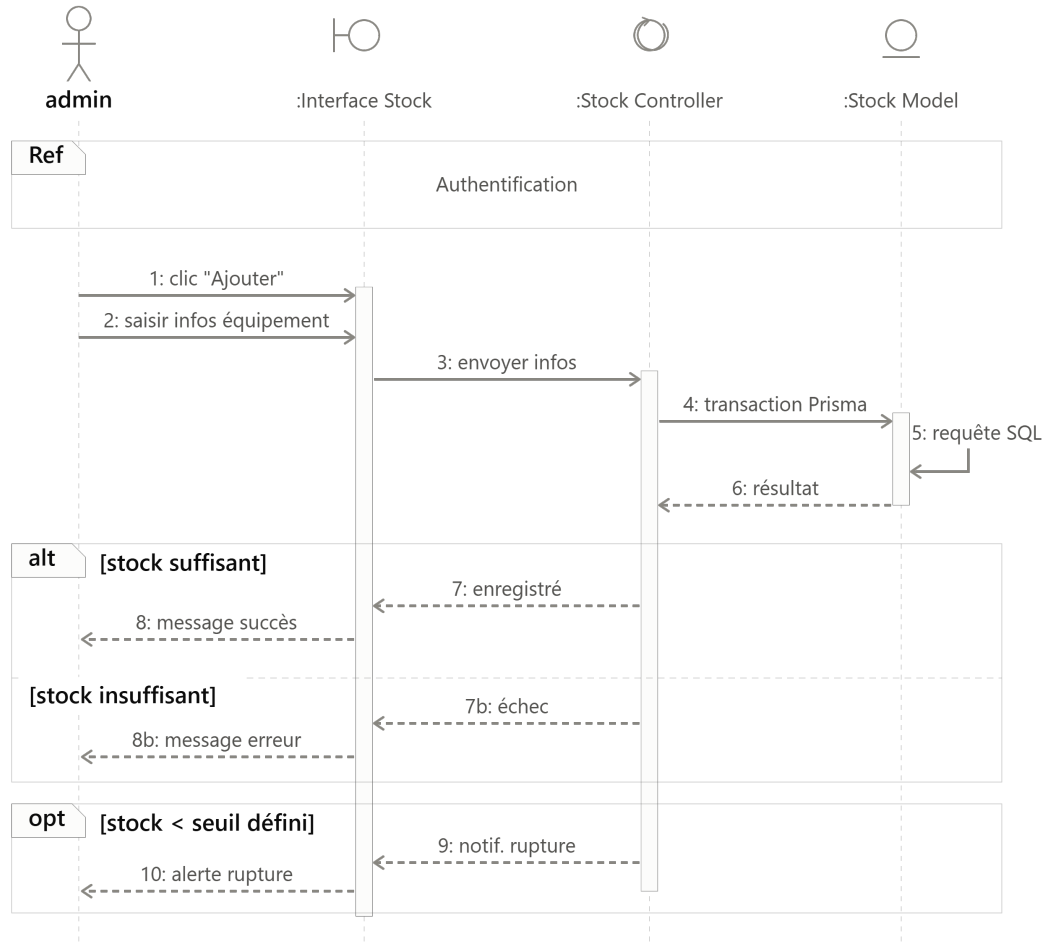


FIGURE 3.9 : Diagramme de séquence – Gérer le Stock

3.6.3.2 Cas d'utilisation «Consulter le Catalogue»

• **Description textuelle** Le tableau 3.5 montre la description du scénario "Consulter le Catalogue des produits".

Cas d'utilisation	Consulter le Catalogue
Acteur	Visiteur, Client Entreprise
Objectif	Permettre à tout utilisateur de parcourir librement le catalogue des équipements informatiques reconditionnés, d'appliquer des filtres de recherche, de consulter les fiches produits détaillées et de configurer un serveur selon ses besoins, sans nécessiter d'authentification.
Précondition	La plateforme est accessible et le catalogue contient au moins un équipement en stock.
Post-condition	L'utilisateur visualise les résultats filtrés, la fiche détaillée du produit sélectionné, ou la configuration personnalisée du serveur.
Scénario nominal	1. L'utilisateur accède à la page catalogue de la plateforme. 2. Le système affiche la liste des équipements disponibles en stock. 3. L'utilisateur applique des filtres (marque, prix, capacité, type). 4. Le système met à jour dynamiquement les résultats affichés. 5. L'utilisateur sélectionne un produit pour consulter sa fiche détaillée ou pour configurer un serveur (RAM, stockage, processeur). 6. Le système affiche les caractéristiques techniques, l'état, le prix et les images du produit ou de la configuration choisie.
Scénario alternatif	A1 Aucun produit ne correspond aux critères de filtrage. A2 Le système affiche un message indiquant l'absence de résultats.
	A3 L'utilisateur peut modifier ses critères et relancer la recherche.

TABLEAU 3.5 : Description textuelle – Cas d'utilisation «Consulter le Catalogue»

• Diagramme de séquence

La figure 3.14 présente le diagramme de séquence du cas d'utilisation Rechercher un Produit, illustrant les interactions entre le visiteur, l'interface Next.js, l'API Route, Prisma ORM et PostgreSQL lors d'une recherche filtrée dans le catalogue

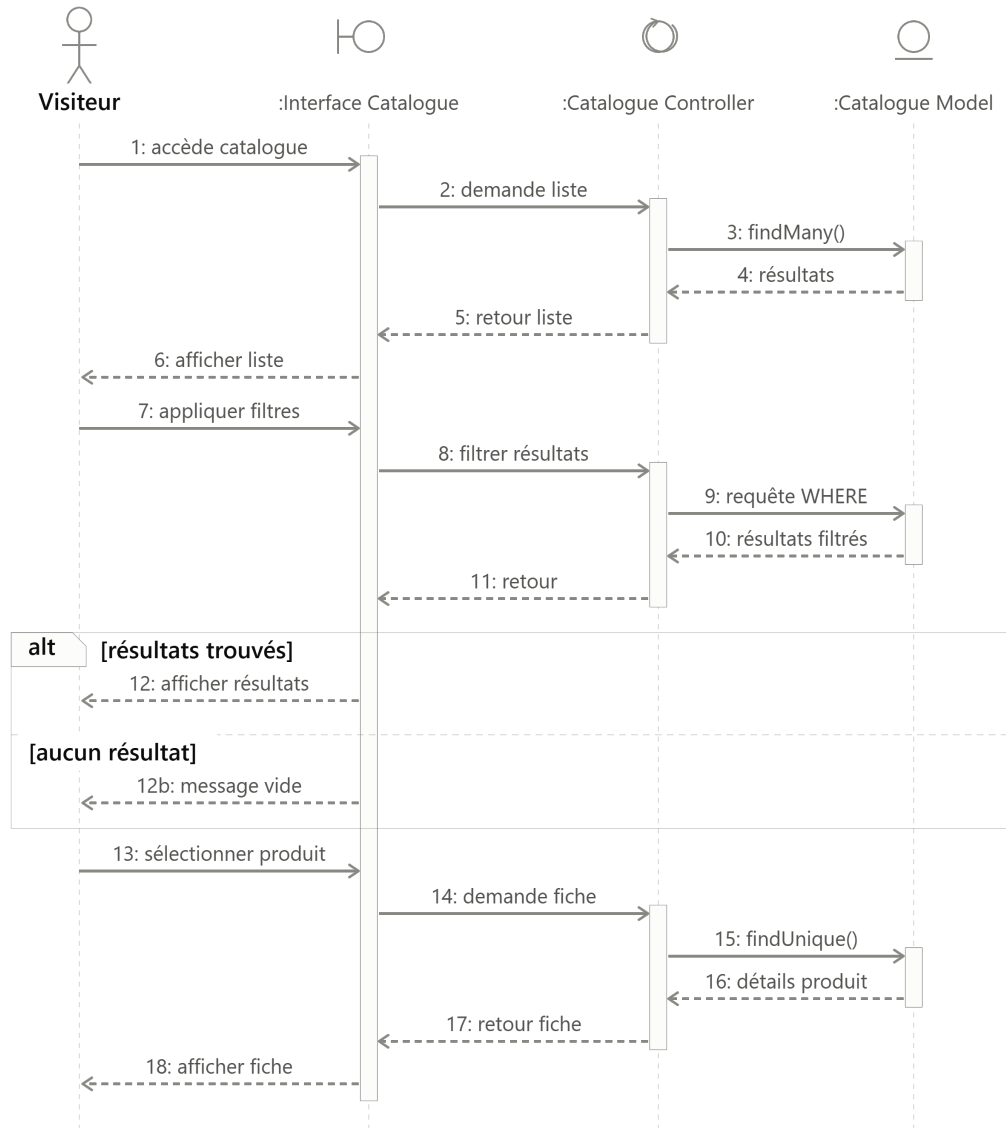


FIGURE 3.10 : Diagramme de séquence – Rechercher un Produit

3.6.4 Réalisation du Sprint 2

Les figures 3.12 et ?? présentent les interfaces implémentées pour le Sprint 2, couvrant la page d'administration du stock et la page catalogue avec filtres.

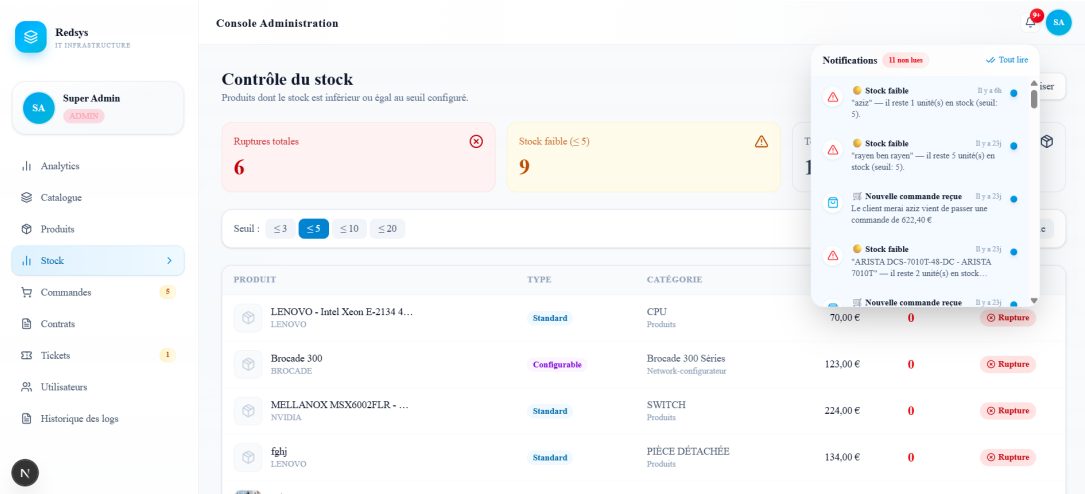


FIGURE 3.11 : Sprint 2 – Interface de Gestion du Stock

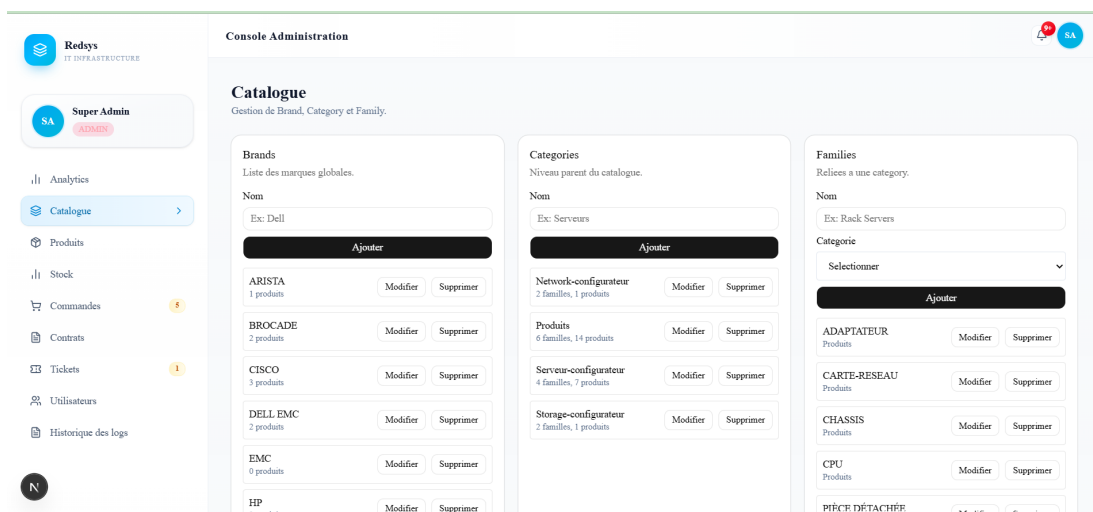


FIGURE 3.12 : Sprint 2 – Interface Catalogue

3.7 Sprint 3 : Panier et Commandes

3.7.1 Objectifs du Sprint 3

L'objectif du Sprint 3 est de livrer le module de gestion du panier et de commande, complétant ainsi le parcours d'achat initié lors du Sprint 2. Les visiteurs pourront ajouter des équipements au panier, modifier les quantités et supprimer des produits, tandis que les clients authentifiés pourront valider leur demande de commande et en suivre l'état.

3.7.2 Diagramme de cas d'utilisation du Sprint 3

La figure 3.13 présente le diagramme de cas d'utilisation du Sprint 3, détaillant les interactions entre le Visiteur et le Client Entreprise avec les fonctionnalités de gestion du panier et de validation des commandes.

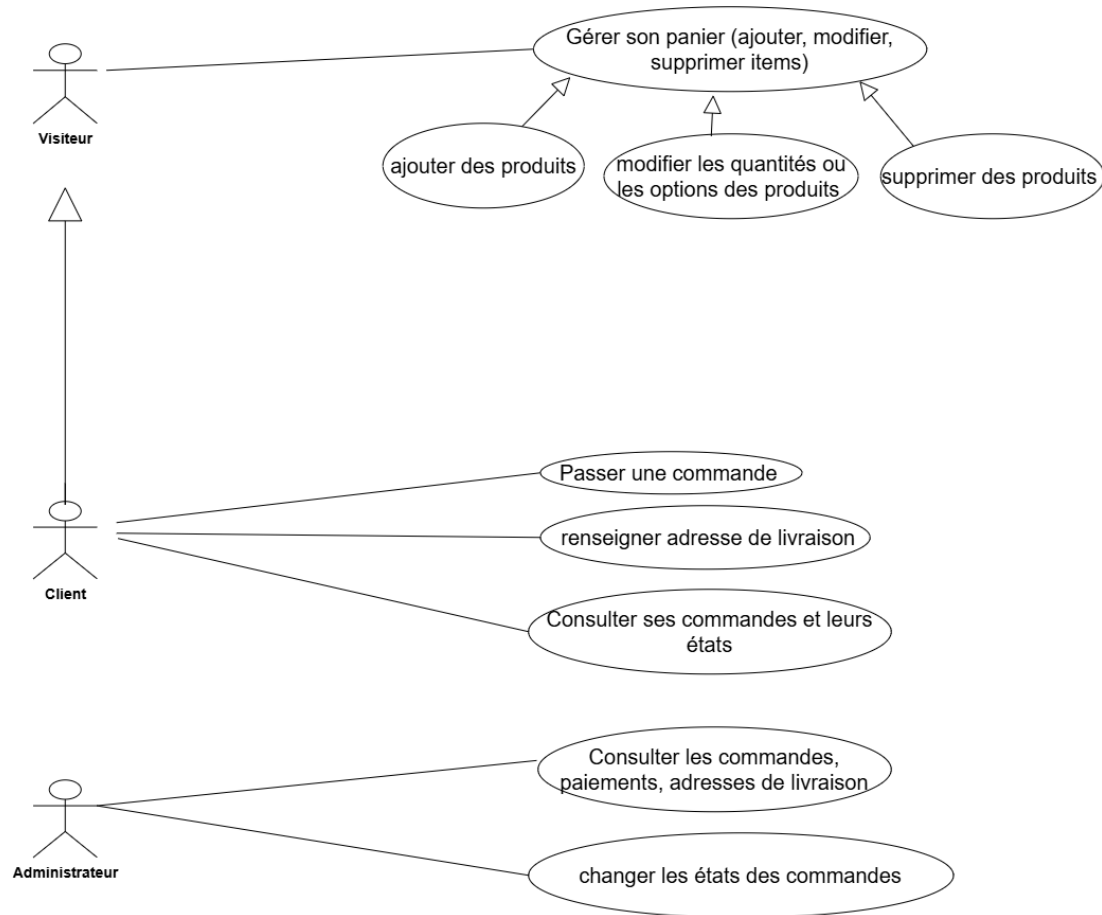


FIGURE 3.13 : Diagramme de cas d'utilisation – Sprint 3

3.7.3 Analyse et Conception du Sprint 3

3.7.3.1 Cas d'utilisation «Gestion du Panier et Commandes»

• Description textuelle

Le tableau 3.6 montre la description du scénario "Gestion du Panier et Commandes".

Cas d'utilisation	Gestion du Panier et Commandes
Acteur	Visiteur, Client Entreprise
Objectif	Permettre à l'utilisateur de gérer son panier (ajout, modification, suppression de produits) et au client authentifié de valider sa demande de commande et d'en suivre l'état.
Précondition	Le catalogue contient des produits disponibles. Pour la validation de commande, le client doit être authentifié.

Post-condition	Les modifications du panier sont sauvegardées. La commande est soumise avec succès et son état est consultable par le client.
Scénario nominal	1. L'utilisateur consulte le catalogue et sélectionne un produit. 2. L'utilisateur ajoute le produit au panier avec la quantité souhaitée. 3. Le système met à jour le récapitulatif du panier (liste, prix total). 4. L'utilisateur consulte son panier et ajuste les quantités ou supprime un produit si nécessaire. 5. Le client authentifié valide sa demande de commande. 6. Le système revalide les prix côté serveur, enregistre la commande et notifie l'administrateur. 7. Le client peut suivre l'état de sa commande depuis son espace.
Scénario alternatif	A1 Le client tente de valider sa commande sans être authentifié. A2 Le système redirige l'utilisateur vers la page de connexion. A3 Après authentification, le panier est conservé et le client reprend à l'étape 5 du scénario nominal.

TABLEAU 3.6 : Description textuelle – Cas d'utilisation «Gestion du Panier et Commandes»

• Diagramme de séquence

La figure 3.14 présente le diagramme de séquence du cas d'utilisation Valider une Commande, illustrant les interactions entre le client, l'interface Next.js, l'API Route, Prisma ORM et PostgreSQL lors du processus de validation de commande, incluant la revalidation serveur des prix et la décrémentation du stock.

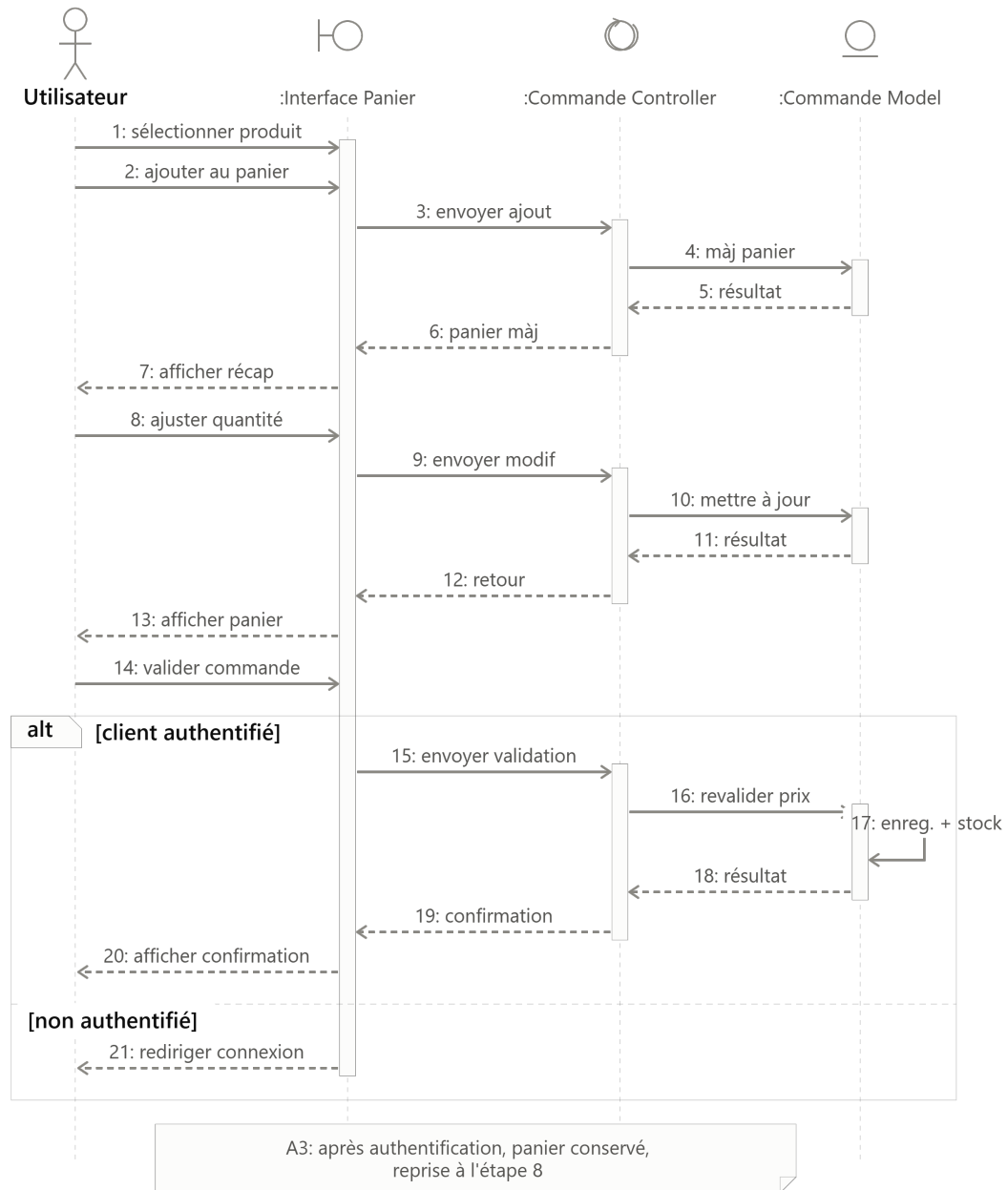


FIGURE 3.14 : Diagramme de séquence – Valider une Commande

3.7.4 Réalisation du Sprint 3

La figure 3.15 présente l'interface implémentée pour le Sprint 3, couvrant la page de gestion du panier et de suivi de commande.

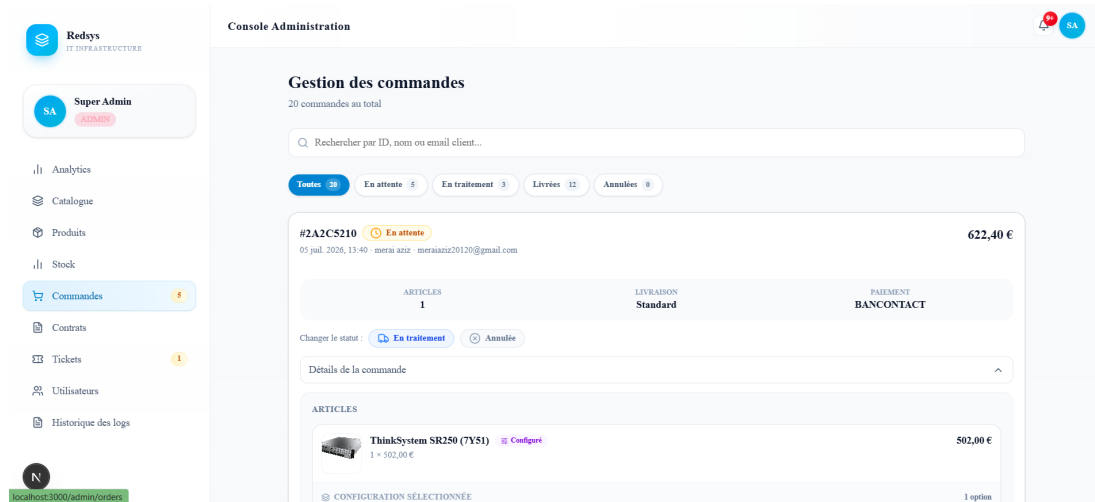


FIGURE 3.15 : Sprint 3 – Interface Panier et Commande

3.8 Conclusion

Ce chapitre a présenté la réalisation complète de la Release 1. Les trois sprints ont livré le socle fondamental de la plateforme : un système d’authentification et de gestion des utilisateurs sécurisé, un module de gestion du stock couplé au catalogue et à la navigation permettant à l’administrateur de piloter les équipements et à l’utilisateur de les découvrir, ainsi qu’un module de panier et de gestion des commandes finalisant le parcours d’achat. Le chapitre suivant présentera la réalisation de la Release 2, qui introduit la gestion des tickets de support, le simulateur de performance, le module de recommandation intelligente basé sur l’IA, ainsi que le tableau de bord analytique.

RELEASE 2 : FONCTIONNALITÉS AVANCÉES

Plan

1	Introduction	60
2	Backlog de la Release 2	60
3	Diagramme de cas d'utilisation de la Release 2	61
4	Diagramme de classes de la Release 2	62
5	Sprint 4 : Gestion des Tickets et Simulation de Performance	63
6	Sprint 5 : Intégration IA et Dashboard Analytique	71
7	Conclusion	77

4.1 Introduction

Dans ce chapitre, nous présenterons les différentes étapes de la deuxième release : le système de gestion des tickets de support couplé au simulateur de performance, ainsi que le module de recommandation intelligente basé sur l'intelligence artificielle couplé au tableau de bord analytique décisionnel.

4.2 Backlog de la Release 2

ID	Fonctionnalité	US ID	User Story	Priorité	Complexité
4	Gestion des Tickets + Simulation de Performance	4.1	En tant que client, je veux soumettre une demande de support (panne, réclamation) via un formulaire dédié.	Must	2
		4.2	En tant que client, je veux suivre l'état de mes tickets (en cours, traité, fermé).	Must	1
		4.3	En tant qu'administrateur, je veux consulter les tickets clients et les affecter aux employés concernés.	Must	2
		4.4	En tant qu'employé, je veux consulter et répondre aux tickets de support qui me sont assignés.	Must	2
		4.5	En tant que client, je veux simuler des scénarios de charge de travail sur un serveur pour évaluer ses capacités.	Must	3
		4.6	En tant que client, je veux visualiser les indicateurs de performance (taux CPU, taux RAM, niveau de risque, stabilité) après simulation.	Must	2
5	Intégration IA + Dashboard Analytique	5.1	En tant que client, je veux saisir mes besoins (nombre d'utilisateurs, type d'application, stockage) pour obtenir une recommandation automatique de serveurs adaptés.	Must	3

Suite à la page suivante...

ID	Fonctionnalité	US ID	User Story	Priorité	Complexité
		5.2	En tant que client, je veux interagir avec un chatbot pour poser des questions sur les produits et obtenir des réponses instantanées.	Should	3
		5.3	En tant que client, je veux recevoir des suggestions personnalisées de produits basées sur mon historique d'achat et mon profil utilisateur.	Should	2
		5.4	En tant qu'administrateur, je veux visualiser les types de serveurs les plus demandés et les configurations les plus recommandées.	Must	3
		5.5	En tant qu'administrateur, je veux filtrer, trier et exporter les données analytiques.	Should	2

TABLEAU 4.1 : Backlog de la Release 2 (Fonctionnalités Avancées)

4.3 Diagramme de cas d'utilisation de la Release 2

La figure 4.1 présente le diagramme de cas d'utilisation raffiné de la Release 2, couvrant le système de ticketing, le simulateur de performance, le module de recommandation intelligente et le tableau de bord analytique.

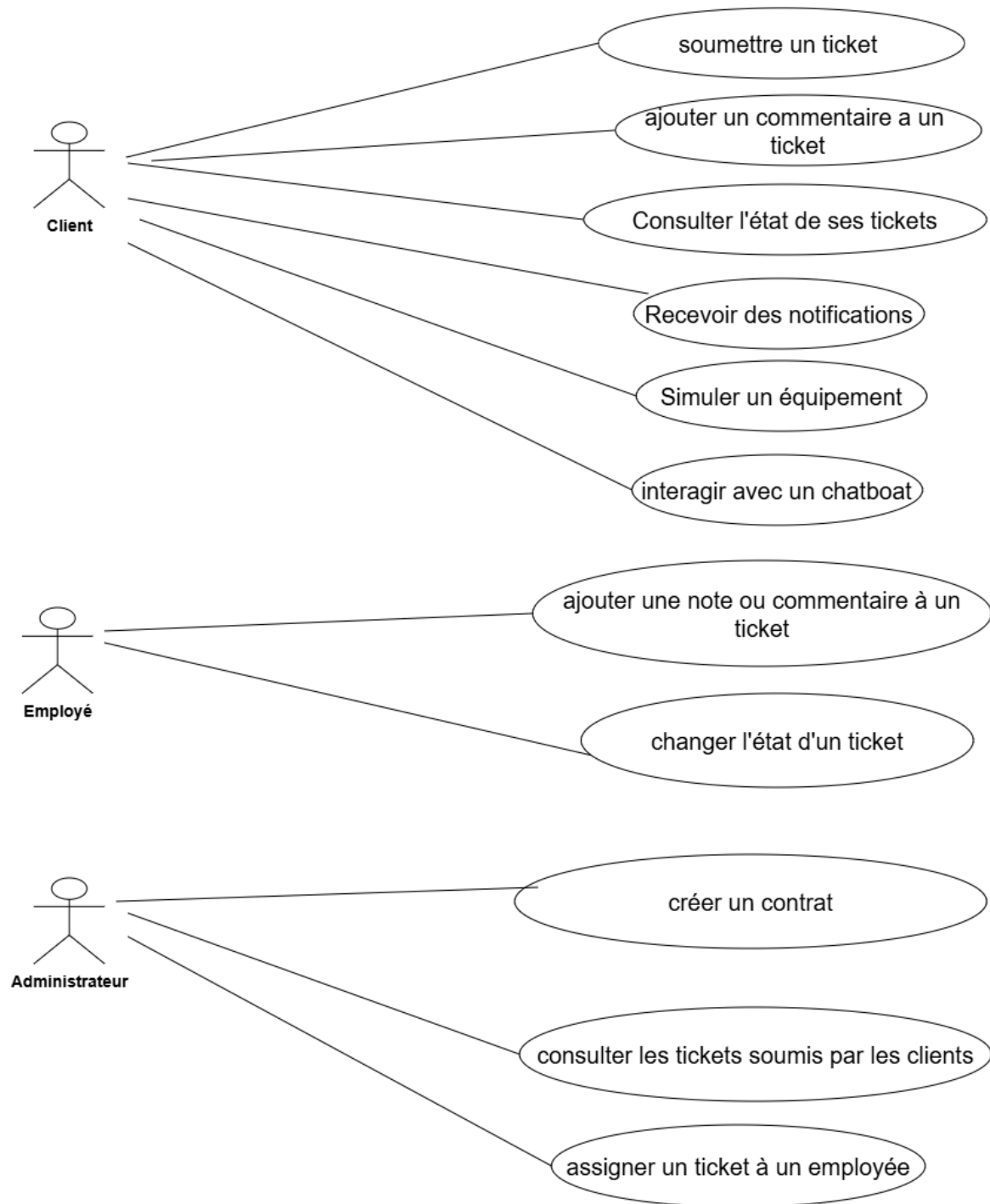


FIGURE 4.1 : Diagramme de cas d'utilisation raffiné – Release 2

4.4 Diagramme de classes de la Release 2

La figure 4.2 présente le diagramme de classes de la Release 2, étendant le modèle de la Release 1 avec les entités introduites dans cette release : Ticket, Message, Simulation, Recommandation et Statistique.

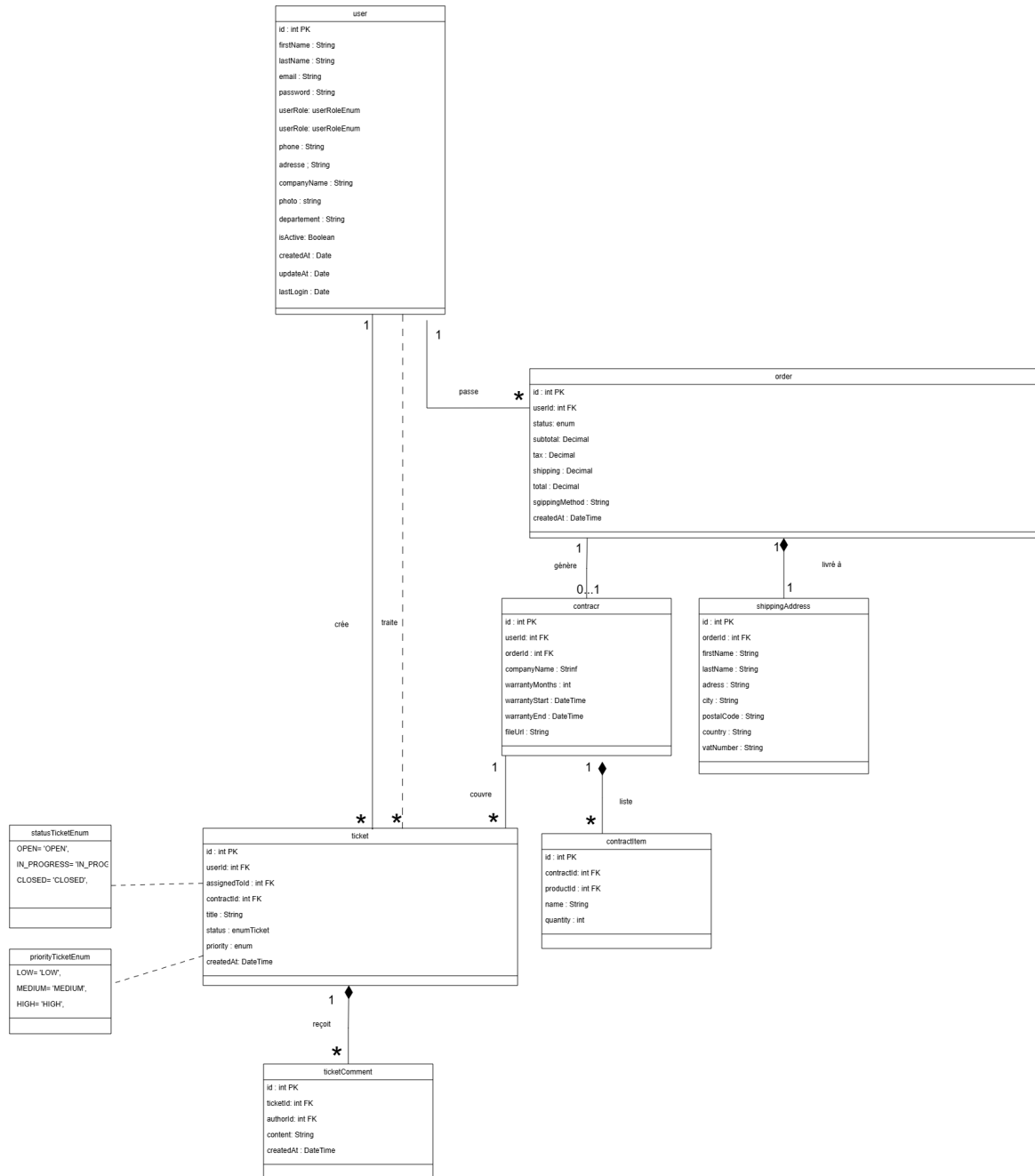


FIGURE 4.2 : Diagramme de classes – Release 2

4.5 Sprint 4 : Gestion des Tickets et Simulation de Performance

4.5.1 Objectifs du Sprint 4

L'objectif du Sprint 4 est de livrer le système de gestion des tickets de support ainsi que le simulateur de performance. Les clients pourront soumettre une demande de support et en suivre l'état, l'administrateur pourra consulter et affecter les tickets aux employés compétents, et les employés pourront répondre aux tickets qui leur sont assignés. En parallèle, les clients pourront simuler des scénarios de charge de travail sur un serveur sélectionné et visualiser les indicateurs de performance calculés.

4.5.2 Diagramme de cas d'utilisation du Sprint 4

La figure 4.3 présente le diagramme de cas d'utilisation raffiné du Sprint 4, détaillant les interactions entre le Client Entreprise, l'Administrateur et l'Employé avec les fonctionnalités de gestion des tickets et de simulation de performance.

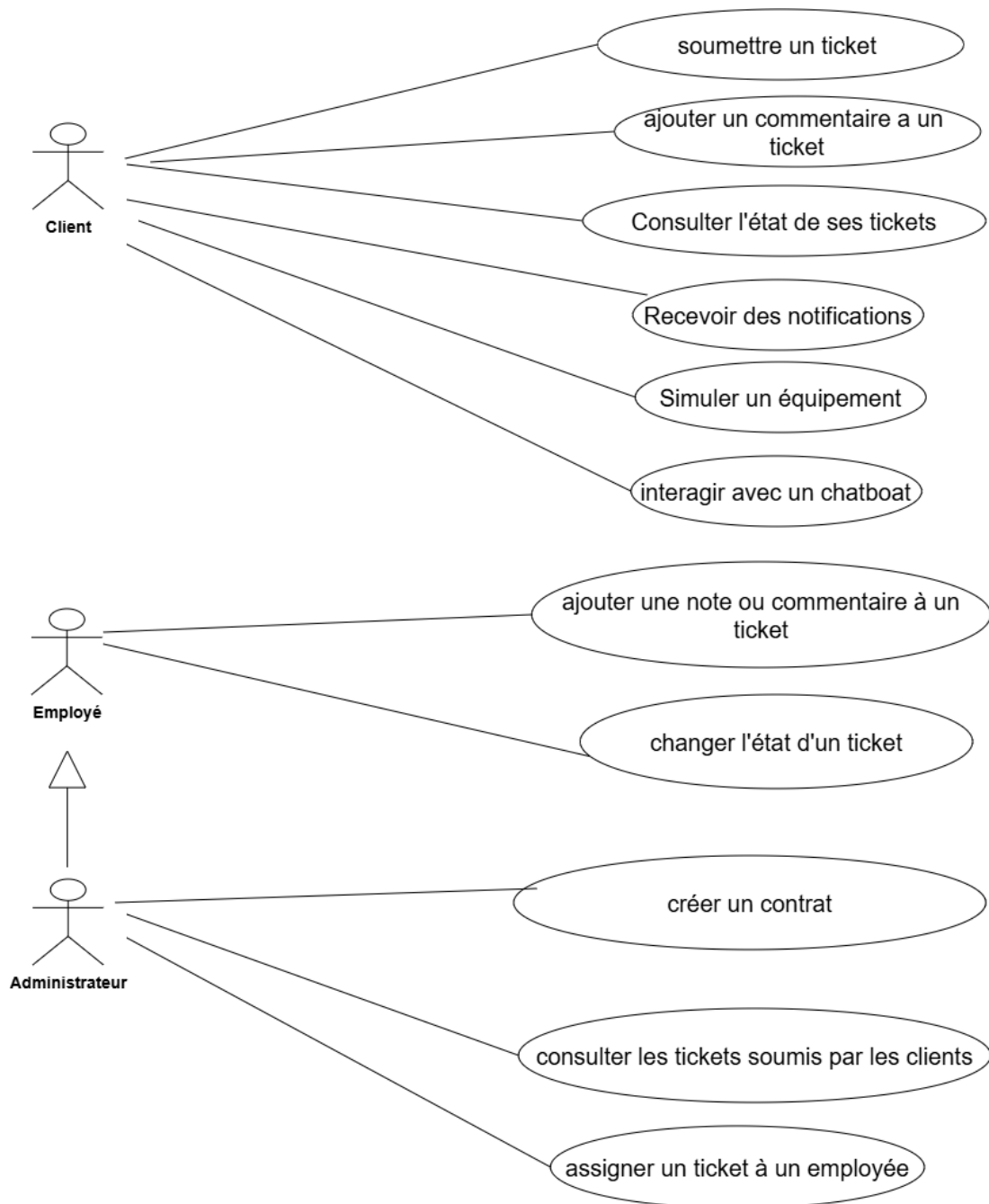


FIGURE 4.3 : Diagramme de cas d'utilisation raffiné – Sprint 4

4.5.3 Analyse et Conception du Sprint 4

4.5.3.1 Cas d'utilisation «Gestion des Tickets de Support»

- Description textuelle

Le tableau 4.2 décrit la description du scénario "Gestion des Tickets de Support".

Cas d'utilisation	Gestion des Tickets de Support
Acteur	Client Entreprise, Administrateur RedSys, Employé RedSys
Objectif	Permettre aux clients de soumettre des demandes de support et d'en suivre l'état, à l'administrateur de consulter et d'affecter les tickets aux employés, et aux employés de répondre aux tickets qui leur sont assignés.
Précondition	Le client, l'administrateur et l'employé doivent être authentifiés avec leurs rôles respectifs.
Post-condition	Le ticket est créé, affecté et traité avec succès. Son statut est mis à jour et consultable par le client à tout moment.
Scénario nominal	1. Le client soumet une demande de support via le formulaire dédié (panne, réclamation). 2. Le système crée le ticket et notifie l'administrateur. 3. L'administrateur consulte le ticket et l'affecte à un employé compétent. 4. L'employé reçoit la notification et consulte le ticket assigné. 5. L'employé traite la demande et rédige une réponse. 6. Le système met à jour le statut du ticket et notifie le client. 7. Le client consulte la réponse et peut clôturer le ticket.
Scénario alternatif	A1 Aucun employé n'est disponible pour traiter le ticket. A2 L'administrateur conserve le ticket en attente d'affectation. A3 L'administrateur peut traiter directement le ticket ou le réaffecter ultérieurement.

TABLEAU 4.2 : Description textuelle – Cas d'utilisation «Gestion des Tickets de Support»

• Diagramme de séquence

La figure 4.4 présente le diagramme de séquence du cas d'utilisation Soumettre un Ticket, illustrant les interactions entre le client, l'interface Next.js, l'API Route, Prisma ORM, PostgreSQL et le système de notification lors du processus de soumission et d'affectation d'un ticket.

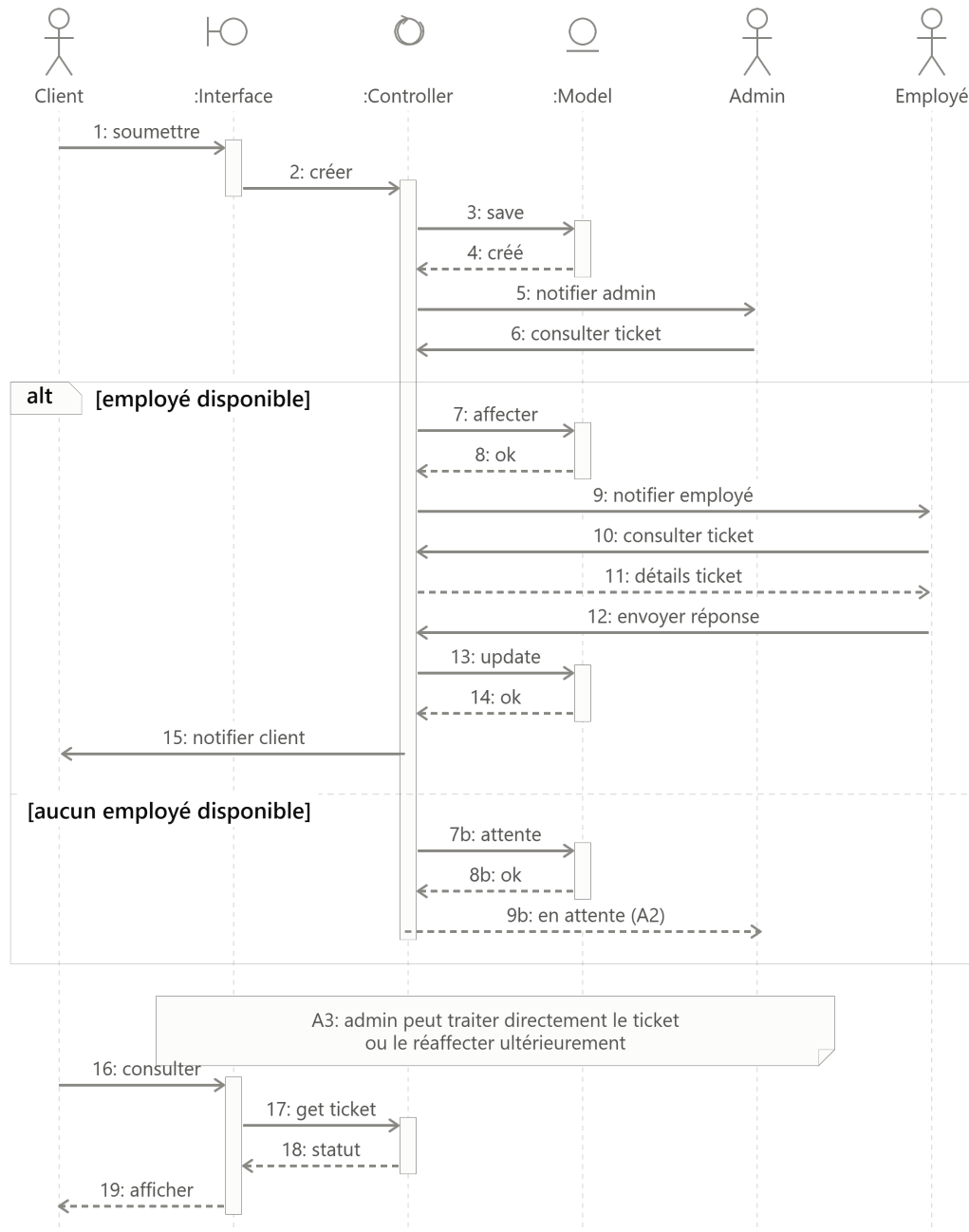


FIGURE 4.4 : Diagramme de séquence – Soumettre un Ticket

4.5.3.2 Cas d'utilisation «Simulation de Performance»

• Description textuelle

Cas d'utilisation	Simulation de Performance
Acteur	Client Entreprise
Objectif	Permettre au client de simuler des scénarios de charge de travail sur un serveur sélectionné et de visualiser les indicateurs de performance calculés.
Précondition	Le client doit être authentifié et avoir sélectionné un équipement à simuler.
Post-condition	Le client visualise les indicateurs de performance (taux CPU, taux RAM, niveau de risque, stabilité) pour le scénario simulé.
Scénario nominal	1. Le client sélectionne un équipement et accède au simulateur. 2. Le système affiche le formulaire de configuration du scénario. 3. Le client définit les paramètres de simulation (charge, nombre d'utilisateurs simultanés). 4. Le système calcule les indicateurs de performance. 5. Le système affiche les résultats (taux CPU, RAM, niveau de risque, indicateur de stabilité). 6. Le client peut modifier les paramètres et relancer une simulation.
Scénario alternatif	A1 Les paramètres de simulation saisis sont invalides. A2 Le système affiche un message d'erreur de validation. A3 Le client corrige les paramètres et relance la simulation.

TABLEAU 4.3 : Description textuelle – Cas d'utilisation «Simulation de Performance»

• Diagramme de séquence

La figure 4.5 présente le diagramme de séquence du cas d'utilisation Simuler une Charge, illustrant les interactions entre le client, l'interface Next.js, l'API Route, le service IA externe (FastAPI/Python)

et PostgreSQL.

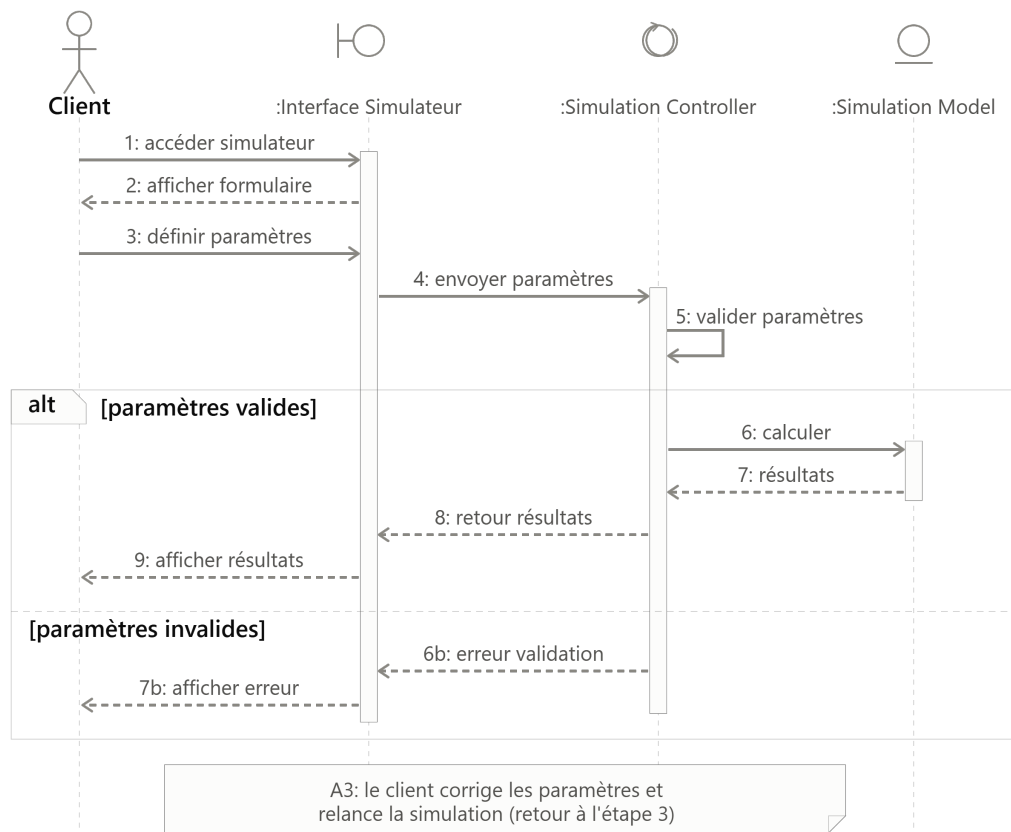
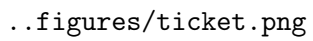


FIGURE 4.5 : Diagramme de séquence – Simuler une Charge

4.5.4 Réalisation du Sprint 4

Les figures 4.6 et 4.7 présentent les interfaces implémentées pour le Sprint 4, couvrant l'interface de gestion des tickets de support et le simulateur de performance.

The image is a placeholder for a screenshot of the Support Ticket Management Interface. It contains the text `..figures/ticket.png`, which likely refers to the file path of the screenshot in the project's figure directory.

`..figures/ticket.png`

FIGURE 4.6 : Sprint 4 – Interface de Gestion des Tickets de Support

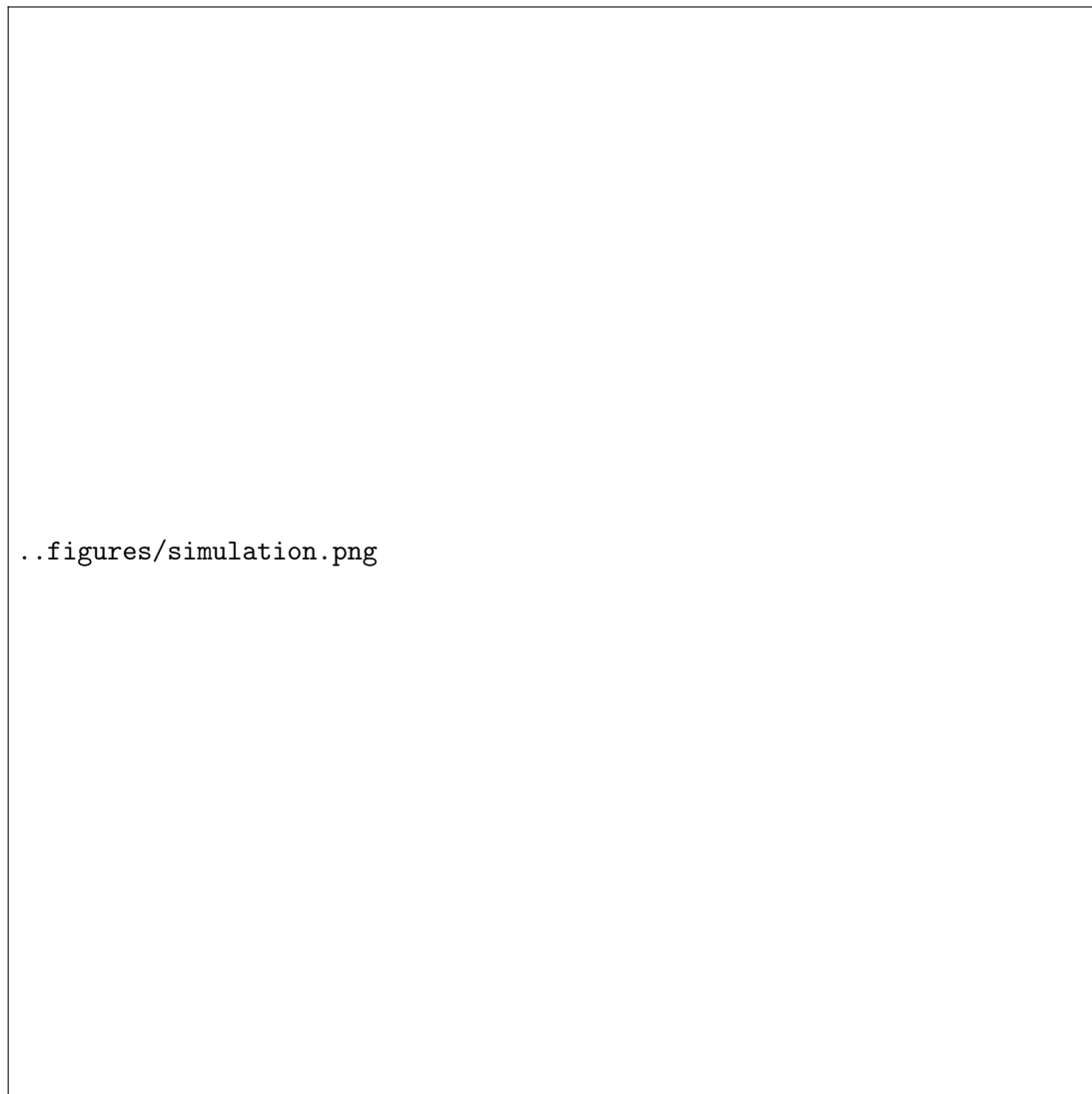


FIGURE 4.7 : Sprint 4 – Interface de Simulation de Performance

4.6 Sprint 5 : Intégration IA et Dashboard Analytique

4.6.1 Objectifs du Sprint 5

L'objectif du Sprint 5 est de livrer le module de recommandation intelligente couplé au tableau de bord analytique décisionnel. Les clients pourront saisir leurs besoins et obtenir une recommandation automatique de serveurs via le service IA externe développé en Python/FastAPI, interagir avec un chatbot et recevoir des suggestions personnalisées basées sur leur historique. En parallèle, l'administrateur pourra visualiser les indicateurs analytiques de la plateforme et exporter les données pour analyse.

4.6.2 Diagramme de cas d'utilisation du Sprint 5

La figure 4.8 présente le diagramme de cas d'utilisation raffiné du Sprint 5, détaillant les interactions entre le Client Entreprise, l'Administrateur et le service IA externe avec les fonctionnalités de recommandation intelligente et de dashboard analytique.



FIGURE 4.8 : Diagramme de cas d'utilisation raffiné – Sprint 5

4.6.3 Analyse et Conception du Sprint 5

4.6.3.1 Cas d'utilisation «Recommandation Intelligente»

- Description textuelle

Cas d'utilisation	Recommandation Intelligente
Acteur	Client Entreprise
Objectif	Permettre au client de saisir ses besoins et d'obtenir automatiquement une recommandation de serveurs reconditionnés adaptés, générée par le service IA externe, avec le score de performance estimé, tout en pouvant interagir avec un chatbot et recevoir des suggestions personnalisées basées sur son historique d'achat.
Précondition	Le client doit être authentifié. Le service IA externe (Python/FastAPI) doit être opérationnel et accessible.
Post-condition	Le client reçoit une liste de serveurs recommandés avec leurs caractéristiques (RAM, stockage, score de performance) et peut les ajouter directement au panier.
Scénario nominal	1. Le client accède au module de recommandation. 2. Le système affiche le formulaire de saisie des besoins. 3. Le client renseigne ses besoins (nombre d'utilisateurs, type d'application, besoin en stockage). 4. L'API Route Next.js transmet la requête au service IA externe. 5. Le service IA analyse le profil et génère les recommandations, en tenant compte de l'historique d'achat du client. 6. Le système affiche les serveurs recommandés avec leurs détails et scores de performance. 7. Le client peut interroger un chatbot pour obtenir des précisions sur un produit recommandé. 8. Le client peut ajouter un équipement recommandé directement au panier.
Scénario alternatif	A1 Le service IA externe est indisponible. A2 Le système affiche un message d'indisponibilité

- **Diagramme de séquence**

La figure 4.9 présente le diagramme de séquence du cas d'utilisation Obtenir une Recommandation, illustrant les interactions entre le client, l'interface Next.js, l'API Route, le service IA externe (FastAPI/Python), Prisma ORM et PostgreSQL.



FIGURE 4.9 : Diagramme de séquence – Obtenir une Recommandation

4.6.3.2 Cas d'utilisation «Dashboard Analytique»

- **Description textuelle**

Cas d'utilisation	Dashboard Analytique
Acteur	Administrateur RedSys
Objectif	Permettre à l'administrateur de visualiser les indicateurs analytiques de la plateforme : serveurs les plus demandés, configurations recommandées les plus fréquentes, et d'exporter les données pour analyse externe.
Précondition	L'administrateur doit être authentifié. Des données de recommandation et d'activité doivent exister dans la base de données.
Post-condition	L'administrateur dispose d'une vue analytique actualisée lui permettant de prendre des décisions stratégiques sur le catalogue et les recommandations proposées.
Scénario nominal	1. L'administrateur accède au tableau de bord analytique. 2. Le système interroge PostgreSQL via Prisma pour agréger les données de recommandation et d'activité. 3. Le dashboard affiche les indicateurs clés (serveurs les plus demandés, configurations les plus recommandées). 4. L'administrateur applique des filtres ou trie les données selon ses besoins. 5. L'administrateur exporte les données analytiques si nécessaire.
Scénario alternatif	A1 Aucune donnée n'est disponible pour la période sélectionnée. A2 Le système affiche un message indiquant l'absence de données. A3 L'administrateur peut modifier la période d'analyse et relancer la requête.

TABLEAU 4.5 : Description textuelle – Cas d'utilisation «Dashboard Analytique»

- **Diagramme de séquence**

La figure 4.10 présente le diagramme de séquence du cas d'utilisation Consulter le Dashboard, illustrant les interactions entre l'administrateur, l'interface Next.js, l'API Route, Prisma ORM et PostgreSQL lors de la récupération et de l'affichage des indicateurs analytiques.



FIGURE 4.10 : Diagramme de séquence – Consulter le Dashboard Analytique

4.6.4 Réalisation du Sprint 5

Les figures 4.11 et 4.12 présentent les interfaces implémentées pour le Sprint 5, couvrant le module de recommandation intelligente et le tableau de bord analytique.

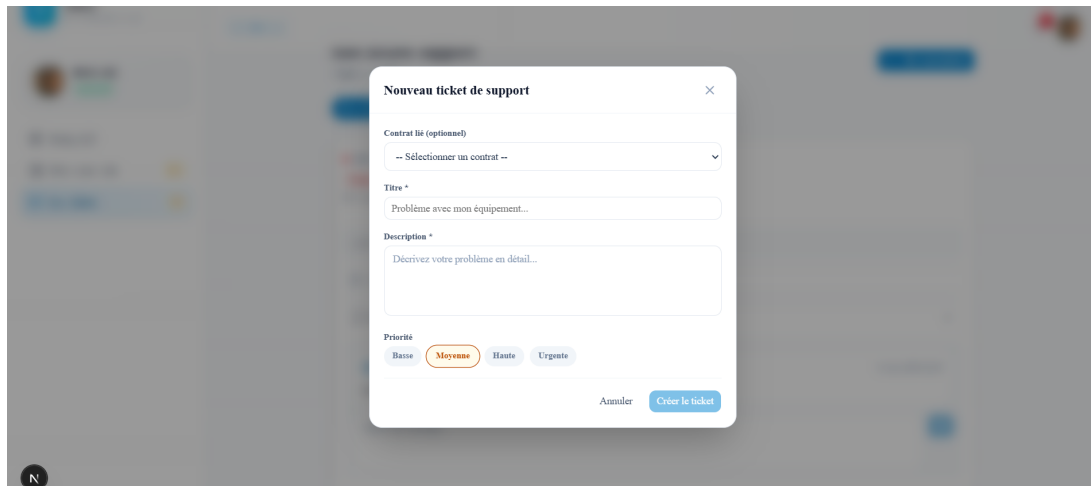


FIGURE 4.11 : Sprint 5 – Interface de Recommandation Intelligente

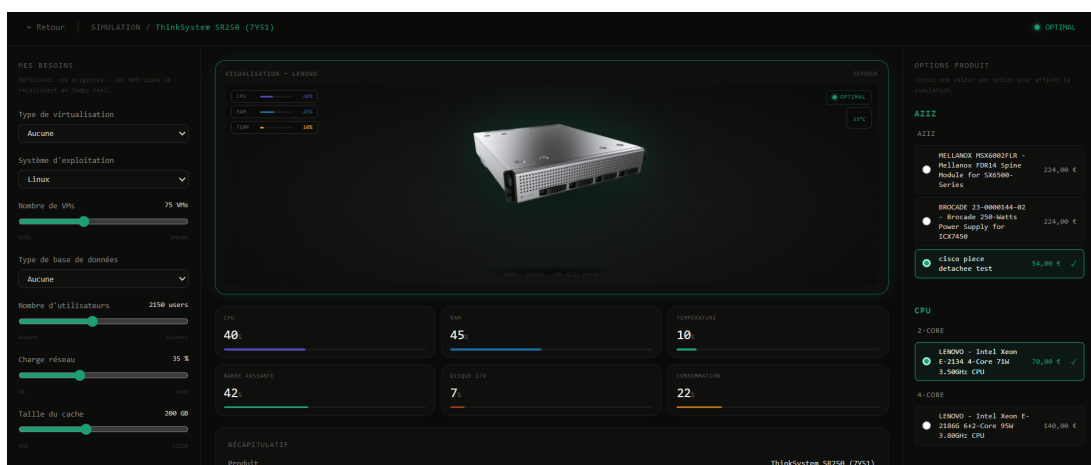


FIGURE 4.12 : Sprint 5 – Tableau de Bord Analytique

4.7 Conclusion

Ce chapitre a présenté la réalisation complète de la Release 2. Le Sprint 4 a livré le système de gestion des tickets de support, offrant à RedSys un outil structuré de relation client, ainsi que le simulateur de performance permettant aux clients d'évaluer les capacités des serveurs avant l'achat. Le Sprint 5 a introduit le module de recommandation intelligente basé sur un service IA externe, couplé à un chatbot et à des suggestions personnalisées, ainsi que le tableau de bord analytique permettant à l'administrateur de piloter son activité commerciale. Les deux releases constituent ensemble une plateforme intelligente, complète et évolutive, positionnant RedSys comme un acteur innovant dans la valorisation des infrastructures IT reconditionnées.

Conclusion Générale

This report presents our end-of-studies project carried out within the framework of our internship at **EntityService**, as part of our academic journey at **Tek-Up University** in computer engineering. This project offered us the opportunity to apply the theoretical and technical knowledge acquired throughout our studies, particularly in the development of modern, integrated, and real-time software systems.

Our project, entitled «*Conception et développement d’une plateforme de gestion et de supervision en temps réel des bornes de recharge pour véhicules électriques*», responds to a growing operational need : replacing a fragmented, spreadsheet-based workflow with a reliable, scalable, and professional platform capable of managing EV charging station data and technician missions in real time.

Through the development of this platform, we designed and implemented an integrated system composed of three core components : an automated data collector that continuously ingests real-time measurements from EV charging stations, a cross-platform mobile application dedicated to field technicians for managing their intervention missions, and a web supervision dashboard providing back-office teams with a comprehensive, real-time view of the entire station fleet.

Throughout this project, we adopted an agile development approach based on the **Scrum** methodology, enabling incremental and iterative delivery of the system across clearly defined sprints. This approach helped us structure our work into two releases and five sprints, each delivering functional and testable increments of the platform.

During our internship at EntityService, we successfully realized the following modules :

- Secure authentication and role-based user management for administrators, supervisors, and technicians ;
- Mission management with real-time lifecycle tracking and assignment to field technicians ;
- A cross-platform mobile application with offline mode and automatic synchronization ;
- An automated data collector service for continuous EV station data ingestion, validation, and storage ;
- A web supervision dashboard with real-time station monitoring, configurable alerts, and data visualization.

Thanks to these achievements, the platform enables EntityService’s client to significantly improve the supervision of their station fleet, reduce human error, gain operational traceability, and enhance the productivity of both field and back-office teams. Furthermore, the integration of a full observability stack combining **ELK**, **Prometheus**, and **Grafana** ensures efficient monitoring and prepares the

platform for future scaling.

However, this project represents a first version that opens the way to several possible evolutions. Among the planned improvements are the completion of the multi-tenant SaaS architecture enabling the management of multiple independent station fleets under a single platform instance, the integration of AI-based anomaly detection for proactive maintenance, and the addition of predictive maintenance suggestions based on historical time-series data collected by the platform.

In summary, this project constitutes a solid and innovative foundation for addressing the current challenges of EV charging station management. It delivers a reliable, scalable, and future-ready architecture, and offers a promising framework for further developments aimed at accompanying the digital transformation of the electric mobility sector.

Webography

- [1] W3C, « HTML5 Specification », World Wide Web Consortium, rapp. tech., 2014. adresse : <https://www.w3.org/TR/html5>
- [2] W3C, « CSS Snapshot », World Wide Web Consortium, rapp. tech., 2023. adresse : <https://www.w3.org/TR/css-2023>
- [3] MICROSOFT, *TypeScript : JavaScript With Syntax For Types*, 2012. adresse : <https://www.typescriptlang.org>
- [4] META PLATFORMS, *React : A JavaScript Library for Building User Interfaces*, 2013. adresse : <https://react.dev>
- [5] POSTGRESQL GLOBAL DEVELOPMENT GROUP, *PostgreSQL : The World's Most Advanced Open Source Database*, 1996. adresse : <https://www.postgresql.org>
- [6] DOCKER INC., *Docker : Accelerated Container Application Development*, 2013. adresse : <https://www.docker.com>
- [7] GITHUB INC., *GitHub : Where the World Builds Software*, 2008. adresse : <https://github.com>
- [8] MICROSOFT, *Visual Studio Code*, 2015. adresse : <https://code.visualstudio.com>
- [9] POSTMAN INC., *Postman API Platform*, 2012. adresse : <https://www.postman.com>
- [10] LINUS TORVALDS, *Git : Distributed Version Control System*, 2005. adresse : <https://git-scm.com>

Résumé

Ce rapport présente le travail effectué dans le cadre du projet de fin d'études à Tek-Up University, réalisé au sein d'EntityService.

L'objectif est la conception et le développement d'une plateforme intégrée de gestion et supervision en temps réel des bornes de recharge pour véhicules électriques. La plateforme comprend trois composants : un collecteur automatique de données, une application mobile pour les techniciens de terrain, et un tableau de bord web de supervision.

Le système repose sur une architecture moderne combinant un backend en **Go** avec API **GraphQL**, un dashboard **React**, une application **React Native**, et une stack d'observabilité (**ELK**, **Prometheus**, **Grafana**). Le projet a été mené selon la méthodologie **Scrum**, organisée en deux releases et cinq sprints.

Mots clés : Véhicule Électrique, Borne de Recharge, Supervision Temps Réel, Go, GraphQL, React, Scrum

Abstract

This report summarizes the end-of-studies project for the computer engineering diploma at Tek-Up University, completed at EntityService.

The objective is the design and development of an integrated platform for real-time management and supervision of electric vehicle charging stations. The platform consists of three components: an automatic data collector, a mobile app for field technicians, and a web supervision dashboard. The system is built on a modern architecture combining a **Go** backend with **GraphQL** API, a **React** dashboard, a **React Native** mobile app, and an observability stack (**ELK**, **Prometheus**, **Grafana**). The project followed **Scrum** agile methodology, organized into two releases and five sprints.

Keywords : Electric Vehicle, Charging Station, Real-Time Supervision, Go, GraphQL, React, React Native, Scrum
